

Divide and Conquer — Complete Guide

A comprehensive guide for SDE placements, online assessments, and competitive programming.

Merge Sort

1. Overview

Merge Sort is a **divide-and-conquer** sorting algorithm that splits an array into two halves, recursively sorts each half, and then merges the two sorted halves into one. It is one of the most efficient comparison-based sorting algorithms with a guaranteed $O(n \log n)$ worst-case time.

2. Intuition

Simple explanation:

Break a large unsorted list into tiny pieces (single elements), then repeatedly merge the pieces back together, but in sorted order.

Analogy:

Imagine you have a deck of cards scattered on the floor. Instead of sorting the whole deck at once, you pick up pairs of cards, sort each pair, then combine sorted pairs into sorted sets of four, then eight, and so on — until the whole deck is sorted.

Step-by-step reasoning:

1. If the array has 0 or 1 elements, it's already sorted — return.
2. Split the array into two halves.
3. Recursively sort both halves.
4. Merge the two sorted halves into one sorted array.

Why it works:

The merge step is the key. Given two already-sorted arrays, you can merge them in $O(n)$ time by always picking the smaller front element. By applying this bottom-up, the entire array ends up sorted.

3. When to Use It

- When you need **stable sorting** (preserves relative order of equal elements).
- When you need **guaranteed $O(n \log n)$** performance (unlike Quick Sort's worst-case).
- When sorting **linked lists** (Merge Sort is the best choice for linked lists).
- When you need to sort **external data** (too large to fit in memory — Merge Sort is the basis for external sorting).
- When you need to count **inversions** in an array.

Common trigger phrases: "sort in $O(n \log n)$ ", "stable sort", "sort linked list", "count inversions", "external sort".

4. When Not to Use It

- When **in-place sorting** is required (Merge Sort uses $O(n)$ extra space).
- When sorting **small arrays** (Insertion Sort is faster due to lower overhead).
- When **memory is limited** (the extra $O(n)$ space can be a problem).
- When you need **fast average-case performance** and memory is tight — Quick Sort is usually preferred.
- When the data is **nearly sorted** — Insertion Sort or Tim Sort would be more efficient.

5. Core Concepts

Divide Step

- Split the array into two halves at the middle index.
- Time: $O(1)$ — just computing `mid = (l + r) / 2`.

Conquer Step

- Recursively sort the two halves.
- This is where the divide-and-conquer recursion happens.

Merge Step

- Combine two sorted subarrays into one sorted array.
- Uses a temporary array to hold the merged result.
- Core operation: compare the front elements of both subarrays and pick the smaller one.

Merge Function

```
merge(arr, left, mid, right):
    copy arr[left..mid] into L[]
    copy arr[mid+1..right] into R[]
    i = j = 0, k = left
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: arr[k++] = L[i++]
        else: arr[k++] = R[j++]
    copy remaining elements from L or R
```

6. Step-by-Step Algorithm

```
mergeSort(arr, left, right):
    1. if left >= right, return (base case)
    2. mid = (left + right) / 2
    3. mergeSort(arr, left, mid)      // sort left half
    4. mergeSort(arr, mid + 1, right) // sort right half
    5. merge(arr, left, mid, right)  // merge sorted halves
```

7. Dry Run

Input: [38, 27, 43, 3, 9, 82, 10]

```
Step 1: Split [38, 27, 43, 3, 9, 82, 10]
        left = [38, 27, 43, 3], right = [9, 82, 10]

Step 2: Split left [38, 27, 43, 3]
        left = [38, 27], right = [43, 3]

Step 3: Split [38, 27]
        left = [38], right = [27] → both single, merge → [27, 38]

Step 4: Split [43, 3]
        left = [43], right = [3] → both single, merge → [3, 43]

Step 5: Merge [27, 38] and [3, 43]
        Compare: 27 vs 3 → pick 3
        Compare: 27 vs 43 → pick 27
        Compare: 38 vs 43 → pick 38
        Pick 43 → [3, 27, 38, 43]

Step 6: Split right [9, 82, 10]
        left = [9], right = [82, 10]

Step 7: Split [82, 10]
        left = [82], right = [10] → merge → [10, 82]

Step 8: Merge [9] and [10, 82]
        Compare: 9 vs 10 → pick 9
        Pick 10, then 82 → [9, 10, 82]

Step 9: Merge [3, 27, 38, 43] and [9, 10, 82]
        Compare: 3 vs 9 → pick 3
        Compare: 27 vs 9 → pick 9
        Compare: 27 vs 10 → pick 10
        Compare: 27 vs 82 → pick 27
        Compare: 38 vs 82 → pick 38
        Compare: 43 vs 82 → pick 43
        Pick 82 → [3, 9, 10, 27, 38, 43, 82]
```

Step	Array
Start	[38, 27, 43, 3, 9, 82, 10]
After merge [38] & [27]	[27, 38, 43, 3, 9, 82, 10]
After merge [43] & [3]	[27, 38, 3, 43, 9, 82, 10]
After merge left half	[3, 27, 38, 43, 9, 82, 10]
After merge [82] & [10]	[3, 27, 38, 43, 9, 10, 82]

Step	Array
Final	[3, 9, 10, 27, 38, 43, 82]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    vector<int> L(n1), R(n2);

    // Copy data to temp arrays
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    // Merge the temp arrays back
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }

    // Copy remaining elements
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Example usage
int main() {
    vector<int> arr = {38, 27, 43, 3, 9, 82, 10};
    mergeSort(arr, 0, arr.size() - 1);

    for (int x : arr) cout << x << " ";
    // Output: 3 9 10 27 38 43 82
}
```

```
    return 0;
}
```

9. Python Implementation

```
def merge(arr, left, mid, right):
    L = arr[left:mid+1]
    R = arr[mid+1:right+1]

    i = j = 0
    k = left

    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1

    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1

    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1

def merge_sort(arr, left, right):
    if left >= right:
        return

    mid = (left + right) // 2
    merge_sort(arr, left, mid)
    merge_sort(arr, mid + 1, right)
    merge(arr, left, mid, right)

# Example usage
arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr, 0, len(arr) - 1)
print(arr) # [3, 9, 10, 27, 38, 43, 82]
```

10. Code Explanation

Merge function:

- **L** and **R** hold copies of the two sorted subarrays.
- The **while** loop compares the first elements of **L** and **R** and picks the smaller one.
- After one array is exhausted, the remaining elements of the other are copied directly.

MergeSort function:

- Base case: if **left** $\geq$ **right**, the subarray has 0 or 1 element — already sorted.
- Compute **mid** to split the array.
- Recursively sort both halves.
- Merge the sorted halves.

Key detail: **mid** = **left** + (**right** - **left**) / 2 avoids integer overflow compared to (**left** + **right**) / 2.

11. Complexity Analysis

Operation	Complexity
Time (best)	$O(n \log n)$
Time (average)	$O(n \log n)$
Time (worst)	$O(n \log n)$
Space (auxiliary)	$O(n)$
Stability	Stable

12. Common Patterns

Pattern 1: Merge Sort + Inversion Counting

- **How to identify:** Problem asks for "number of inversions" or "count pairs where $i < j$ and $arr[i] > arr[j]$ ".
- **Approach:** When merging, if **R[j] < L[i]**, then all remaining elements in **L** are greater than **R[j]** — count them.
- **Example:** Count Inversions (GFG), Reverse Pairs (LeetCode 493).

Pattern 2: External Sorting

- **How to identify:** Data too large to fit in memory; sorting on disk.
- **Approach:** Sort chunks in memory, then merge them using multi-way merge.
- **Example:** Sort a large file (system design).

Pattern 3: Merge K Sorted Arrays/Lists

- **How to identify:** Multiple sorted arrays that need to be combined.
- **Approach:** Use a min-heap (not Merge Sort directly), but the merge idea is similar.
- **Example:** Merge k Sorted Lists (LeetCode 23).

13. Common Mistakes

- **Off-by-one in merge:** The `mid` index must be included in the left half. Left half: `[left..mid]`, Right half: `[mid+1..right]`.
- **Forgetting to copy remaining elements:** After the main merge loop, one of the subarrays may have leftover elements — must copy them.
- **Using $(\text{left} + \text{right}) / 2$:** Can overflow for large arrays. Use `left + (right - left) / 2`.
- **Modifying the original array during merge:** Must copy to temp arrays first, or you'll overwrite values.
- **Not handling empty arrays:** Check `left >= right` for the base case.

14. Edge Cases

- Empty array (`[]`): should return without changes.
- Single element (`[5]`): already sorted.
- Two elements (`[2, 1]`): should swap correctly.
- Duplicate elements: stability must be preserved (`L[i] <= R[j]` ensures stable sort).
- Already sorted array: still $O(n \log n)$.
- Reverse sorted array: still $O(n \log n)$.
- All equal elements: merge will copy all from L first (if using `<=`).

15. Variations

Bottom-Up (Iterative) Merge Sort

- Starts by merging pairs of adjacent single elements, then pairs of size-2, then size-4, etc.
- Avoids recursion overhead.
- Less intuitive but useful for understanding.

In-Place Merge Sort

- Attempts to merge without extra space using rotation or complex swaps.
- Usually slower and more complex; rarely used in practice.

Natural Merge Sort

- Exploits existing sorted runs in the input.
- Used in Tim Sort (Python's built-in sort).

External Merge Sort

- Used when data is too large for RAM.
- Sorts chunks on disk, then merges them.
- Critical for database systems.

16. Related Algorithms

Algorithm	Difference	When to Choose
Quick Sort	In-place, $O(n \log n)$ average, $O(n^2)$ worst	When memory is tight, average-case matters

Algorithm	Difference	When to Choose
Heap Sort	In-place, $O(n \log n)$ worst, not stable	When $O(1)$ extra space and $O(n \log n)$ worst-case needed
Insertion Sort	$O(n^2)$ worst, $O(n)$ best for nearly sorted	Small arrays, nearly sorted data
Tim Sort	Hybrid of Merge & Insertion Sort	Python/Java's built-in sort
Counting Sort	$O(n+k)$, non-comparison	When range is small

17. Practice Problems

Easy

1. **Merge Sorted Array** — LeetCode 88 → Merge two sorted arrays into one.
2. **Sort an Array** — LeetCode 912 → Implement merge sort to sort an array.

Medium

3. **Count Inversions** — GFG → Count inversions in an array.
4. **Sort List** — LeetCode 148 → Sort a linked list in $O(n \log n)$ time.
5. **Reverse Pairs** — LeetCode 493 → Count pairs where $i < j$ and $arr[i] > 2 * arr[j]$.

Hard

6. **Median of Two Sorted Arrays** — LeetCode 4 → Find median of two sorted arrays using binary search + merge idea.
7. **Count of Smaller Numbers After Self** — LeetCode 315 → Count smaller elements to the right using merge sort with indices.

18. Interview Explanation

"Merge Sort is a divide-and-conquer sorting algorithm. It works by recursively splitting the array into two halves until we reach single elements, then merging them back in sorted order. The merge operation is the key: given two sorted subarrays, we compare their front elements and pick the smaller one, which takes $O(n)$ time. The recurrence $T(n) = 2T(n/2) + O(n)$ gives $O(n \log n)$ time. Merge Sort is stable, uses $O(n)$ extra space, and has guaranteed $O(n \log n)$ performance regardless of input distribution. It's especially useful for sorting linked lists and for problems like counting inversions."

19. Revision Notes

- Divide: split at mid, Conquer: recursively sort, Combine: merge.
- Time: $O(n \log n)$ always.
- Space: $O(n)$ auxiliary.
- Stable (if $\leq$ is used in merge).
- Merge function is the critical part — practice writing it.
- Use `left + (right - left) / 2` to avoid overflow.

- Base case: `left >= right`.

20. Final Cheat Sheet

Aspect	Detail
When to use	Guaranteed $O(n \log n)$, stable sort, linked lists, inversion counting
Main operations	Split ($O(1)$), Merge ($O(n)$), Recursion ($2 * T(n/2)$)
Complexity	Time: $O(n \log n)$, Space: $O(n)$
Key code idea	<code>merge(arr, l, m, r)</code> merges two sorted halves
Edge cases	Empty, single element, duplicates, already sorted
Common mistake	Forgetting to copy remaining elements after merge loop

Quick Sort

1. Overview

Quick Sort is a **divide-and-conquer** sorting algorithm that works by selecting a **pivot** element, partitioning the array around it (elements smaller than pivot go left, larger go right), and then recursively sorting the two partitions. It is one of the most widely used sorting algorithms due to its excellent average-case performance.

2. Intuition

Simple explanation:

Pick any element as a "pivot". Rearrange the array so that everything smaller than the pivot comes before it, and everything larger comes after. Now the pivot is in its final sorted position. Recursively do the same for the left and right sides.

Analogy:

Imagine you're organizing a group photo by height. You pick one person (pivot). Everyone shorter goes to the left side, everyone taller to the right. Now that person is in the correct spot. Do the same for the left group and right group independently.

Step-by-step reasoning:

1. Pick a pivot (last element, first element, random, or median-of-three).
2. Partition: rearrange so that all elements $\leq$ pivot are on the left, all $>$ pivot on the right.
3. Recursively Quick Sort the left partition and the right partition.

Why it works:

After each partition step, the pivot is in its final correct position. The left and right subarrays are smaller problems of the same type. Recursion eventually sorts everything.

3. When to Use It

- When you need **fast average-case sorting** ($O(n \log n)$ with low constant factors).

- When **in-place sorting** is needed ($O(\log n)$ stack space for recursion).
- When **memory is limited** and you can't afford $O(n)$ extra space.
- When you need a **general-purpose sort** with good cache performance.
- When the data is **random enough** that worst-case is unlikely.

Common trigger phrases: "sort in-place", "sort with $O(\log n)$ extra space", "fast sorting", "partition".

4. When Not to Use It

- When **stable sorting** is required (Quick Sort is not stable by default).
- When **worst-case $O(n^2)$ is unacceptable** (e.g., hard real-time systems).
- When sorting **nearly sorted data** with a naive pivot choice (worst-case).
- When the array is **very small** — Insertion Sort is faster.
- When the data contains **many duplicates** — standard Quick Sort degrades (use 3-way partition).

5. Core Concepts

Pivot Selection

- The choice of pivot dramatically affects performance.
- **Naive:** always pick first or last element — $O(n^2)$ for sorted arrays.
- **Random pivot:** good average-case, avoids worst-case with high probability.
- **Median-of-three:** pick median of first, middle, last — more robust.

Partitioning (Lomuto vs Hoare)

Lomuto Partition (simpler, used in basic implementations):

- Pivot = last element.
- `i` tracks the boundary of elements $\leq$ pivot.
- Scan with `j`, swap `arr[i]` and `arr[j]` when `arr[j]  $\leq$  pivot`.
- Finally, swap pivot into position.

Hoare Partition (faster, more efficient):

- Pivot = middle element (or any).
- Two pointers from both ends, swap when `left > pivot` and `right < pivot`.
- 3x fewer swaps on average than Lomuto.

Recursion

- After partitioning, the pivot is at its final position.
- Recursively sort `[left, pivotIndex - 1]` and `[pivotIndex + 1, right]`.

6. Step-by-Step Algorithm

```
quickSort(arr, left, right):
  1. If left >= right, return
  2. pivotIndex = partition(arr, left, right)
  3. quickSort(arr, left, pivotIndex - 1)
```

```

4. quickSort(arr, pivotIndex + 1, right)

partition(arr, left, right):
1. pivot = arr[right]    // Lomuto: pick last element
2. i = left               // i marks the boundary of elements ≤ pivot
3. for j = left to right - 1:
    if arr[j] ≤ pivot:
        swap(arr[i], arr[j])
        i++
4. swap(arr[i], arr[right]) // place pivot in correct position
5. return i

```

7. Dry Run

Input: [10, 7, 8, 9, 1, 5]

Partition 1: pivot = 5 (last element)

```

i=0, j=0: arr[0]=10 > 5, no swap
i=0, j=1: arr[1]=7 > 5, no swap
i=0, j=2: arr[2]=8 > 5, no swap
i=0, j=3: arr[3]=9 > 5, no swap
i=0, j=4: arr[4]=1 ≤ 5, swap(arr[0], arr[4]) → [1, 7, 8, 9, 10, 5], i=1
j=5: end of loop

```

```

swap(arr[1], arr[5]) → [1, 5, 8, 9, 10, 7]
pivotIndex = 1

```

```

Left subarray: [1]      → already sorted
Right subarray: [8, 9, 10, 7]

```

Partition 2: pivot = 7 (last element of right subarray)

```

i=2, j=2: arr[2]=8 > 7, no swap
i=2, j=3: arr[3]=9 > 7, no swap
i=2, j=4: arr[4]=10 > 7, no swap

```

```

swap(arr[2], arr[5]) → [1, 5, 7, 9, 10, 8]
pivotIndex = 2

```

```

Left subarray: []      → nothing
Right subarray: [9, 10, 8]

```

Partition 3: pivot = 8 (last element of subarray)

```

i=3, j=3: arr[3]=9 > 8, no swap
i=3, j=4: arr[4]=10 > 8, no swap

```

```

swap(arr[3], arr[5]) → [1, 5, 7, 8, 10, 9]
pivotIndex = 3

```

Right subarray: [10, 9]

Partition 4: pivot = 9

i=4, j=4: arr[4]=10 > 9, no swap
 swap(arr[4], arr[5]) → [1, 5, 7, 8, 9, 10]

Final: [1, 5, 7, 8, 9, 10]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<int>& arr, int left, int right) {
    int pivot = arr[right]; // Lomuto: pivot is last element
    int i = left;           // boundary of elements ≤ pivot

    for (int j = left; j < right; j++) {
        if (arr[j] <= pivot) {
            swap(arr[i], arr[j]);
            i++;
        }
    }
    swap(arr[i], arr[right]);
    return i;
}

void quickSort(vector<int>& arr, int left, int right) {
    if (left >= right) return;

    int pivotIndex = partition(arr, left, right);
    quickSort(arr, left, pivotIndex - 1);
    quickSort(arr, pivotIndex + 1, right);
}

// Hoare partition (faster alternative)
int partitionHoare(vector<int>& arr, int left, int right) {
    int pivot = arr[left + (right - left) / 2];
    int i = left - 1, j = right + 1;

    while (true) {
        do { i++; } while (arr[i] < pivot);
        do { j--; } while (arr[j] > pivot);
        if (i >= j) return j;
        swap(arr[i], arr[j]);
    }
}

void quickSortHoare(vector<int>& arr, int left, int right) {
    if (left >= right) return;
```

```

    int pivotIndex = partitionHoare(arr, left, right);
    quickSortHoare(arr, left, pivotIndex);
    quickSortHoare(arr, pivotIndex + 1, right);
}

// Example usage
int main() {
    vector<int> arr = {10, 7, 8, 9, 1, 5};
    quickSort(arr, 0, arr.size() - 1);

    for (int x : arr) cout << x << " ";
    // Output: 1 5 7 8 9 10
    return 0;
}

```

9. Python Implementation

```

import random

def partition(arr, left, right):
    pivot = arr[right]
    i = left

    for j in range(left, right):
        if arr[j] <= pivot:
            arr[i], arr[j] = arr[j], arr[i]
            i += 1

    arr[i], arr[right] = arr[right], arr[i]
    return i

def quick_sort(arr, left, right):
    if left >= right:
        return

    # Random pivot to avoid worst-case on sorted input
    rand_idx = random.randint(left, right)
    arr[right], arr[rand_idx] = arr[rand_idx], arr[right]

    pivot_index = partition(arr, left, right)
    quick_sort(arr, left, pivot_index - 1)
    quick_sort(arr, pivot_index + 1, right)

# Example usage
arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr) - 1)
print(arr)  # [1, 5, 7, 8, 9, 10]

```

10. Code Explanation

Partition function (Lomuto):

- Pivot is the last element.
- `i` tracks the position where the next element $\leq$ pivot should go.
- `j` scans the array from left to right.
- When `arr[j] ≤ pivot`, swap it with `arr[i]` and increment `i`.
- After the loop, swap pivot into position `i` — now pivot is in its final place.
- Return `i` (the pivot index).

QuickSort function:

- Base case: `left >= right`.
- Partition the array to get the pivot index.
- Recursively sort the left and right partitions.

Random pivot: Before partitioning, swap a random element to the end. This ensures $O(n \log n)$ expected time.

11. Complexity Analysis

Operation	Complexity
Time (best)	$O(n \log n)$
Time (average)	$O(n \log n)$
Time (worst)	$O(n^2)$
Space (auxiliary, Lomuto)	$O(n)$ worst, $O(\log n)$ average (stack)
Stability	Not stable

Worst case occurs when: Pivot is always the smallest or largest element (e.g., sorted array with first/last pivot).

12. Common Patterns

Pattern 1: Quickselect (k-th smallest/largest)

- **How to identify:** "Find k-th smallest/largest element", "find median", "find top k".
- **Approach:** Partition the array. If pivot index == k, return it. If pivot index > k, recurse on left. Else recurse on right.
- **Complexity:** $O(n)$ average, $O(n^2)$ worst.
- **Example:** Kth Largest Element in an Array (LeetCode 215).

Pattern 2: Dutch National Flag (3-way partition)

- **How to identify:** "Sort colors", "sort 0s, 1s, and 2s", "partition with duplicates".
- **Approach:** Use three pointers to partition into three regions: < pivot, = pivot, > pivot.
- **Example:** Sort Colors (LeetCode 75).

Pattern 3: Partition-based problems

- **How to identify:** "Move all X to one side", "rearrange array with condition".
- **Approach:** Use the partition logic without full sorting.
- **Example:** Move Zeroes (LeetCode 283), Segregate Even and Odd.

13. Common Mistakes

- **Stack overflow with worst-case:** For sorted arrays with naive pivot, recursion depth = $O(n)$. Use random pivot or median-of-three.
- **Wrong pivot index in recursion:** After Lomuto partition, recurse on `[left, pivotIndex-1]` and `[pivotIndex+1, right]`. For Hoare, the pivot may not be at `pivotIndex`.
- **Infinite recursion:** If partition doesn't split the array (e.g., all equal elements), can loop forever. Use 3-way partition.
- **Off-by-one in Hoare:** The `do-while` loops increment/decrement before checking, so `i` and `j` start outside the range.
- **Not handling equal elements:** Lomuto puts all equal elements on the left side, which can cause $O(n^2)$ for all-equal arrays.

14. Edge Cases

- Empty array: return immediately.
- Single element: already sorted.
- Already sorted (ascending): worst-case with first/last pivot.
- Already sorted (descending): worst-case with first/last pivot.
- All equal elements: $O(n^2)$ with Lomuto, $O(n \log n)$ with 3-way partition.
- Large array with few unique values: consider 3-way partition.
- Very large array: recursion depth may exceed stack limit.

15. Variations

3-Way Quick Sort (Dutch National Flag)

- Partitions into three regions: $< \text{pivot}$, $= \text{pivot}$, $> \text{pivot}$.
- Handles duplicates efficiently.
- Used in practice for arrays with many duplicates.

Randomized Quick Sort

- Randomly pick pivot to avoid worst-case.
- $O(n \log n)$ expected time.
- Standard in most libraries.

Dual-Pivot Quick Sort

- Uses two pivots, partitions into three regions.
- Used in Java's `Arrays.sort()` for primitives.
- Reduces the number of comparisons.

Introsort

- Hybrid: Quick Sort + Heap Sort + Insertion Sort.
- Switches to Heap Sort if recursion depth exceeds $O(\log n)$.
- Used in C++ `std::sort` and `std::partial_sort`.

16. Related Algorithms

Algorithm	Difference	When to Choose
Merge Sort	$O(n)$ extra space, stable, $O(n \log n)$ worst	When stability or guaranteed time is needed
Heap Sort	In-place, $O(n \log n)$ worst, not stable, worse cache	When $O(1)$ extra space and $O(n \log n)$ worst-case needed
Insertion Sort	$O(n^2)$ worst, $O(n)$ best, stable	Small arrays, nearly sorted data
Selection Sort	$O(n^2)$, fewer swaps	When write cost is high
Counting Sort	$O(n+k)$, non-comparison	When range is small

17. Practice Problems

Easy

1. **Sort Colors** — LeetCode 75 → Dutch national flag / 3-way partition.
2. **Move Zeroes** — LeetCode 283 → Partition-based rearrangement.

Medium

3. **Kth Largest Element in an Array** — LeetCode 215 → Quickselect.
4. **Sort an Array** — LeetCode 912 → Implement Quick Sort.
5. **K Closest Points to Origin** — LeetCode 973 → Quickselect on distances.

Hard

6. **Find Median from Data Stream** — LeetCode 295 → Quickselect approach (alternative: two heaps).
7. **Kth Smallest Element in a Sorted Matrix** — LeetCode 378 → Binary search on value range, but partition-based approach also works.

18. Interview Explanation

"Quick Sort is a divide-and-conquer sorting algorithm that selects a pivot, partitions the array around it, and recursively sorts the partitions. The partition step rearranges elements so that all elements $\leq$ pivot come before it, and all elements $>$ pivot come after — this puts the pivot in its final position. I prefer Hoare's partition for its efficiency, and I always use a random pivot to avoid the $O(n^2)$ worst-case on

sorted input. On average, Quick Sort runs in $O(n \log n)$ time with $O(\log n)$ stack space. The same partitioning logic also powers Quickselect, which finds the k -th smallest element in $O(n)$ average time."

19. Revision Notes

- Pick pivot → partition → recurse left and right.
- Lomuto (simpler, last element pivot) vs Hoare (faster, middle pivot).
- Random pivot avoids worst-case.
- Not stable, not adaptive.
- Worst-case $O(n^2)$ when pivot is always min or max.
- Average $O(n \log n)$ with low constant factor.
- Quickselect: skip one side, $O(n)$ average.

20. Final Cheat Sheet

Aspect	Detail
When to use	In-place sorting, fast average-case, limited memory
Main operations	Partition ($O(n)$), Recursion ($2 * T(n/2)$ average)
Complexity	Time: $O(n \log n)$ avg, $O(n^2)$ worst; Space: $O(\log n)$ stack
Key code idea	<code>partition(arr, l, r)</code> returns pivot index
Edge cases	Sorted (use random pivot), all equal (use 3-way), single element
Common mistake	Picking first/last pivot on sorted data → $O(n^2)$

Binary Search

1. Overview

Binary Search is an **$O(\log n)$** algorithm for finding a target value in a **sorted array**. It works by repeatedly dividing the search interval in half. It is one of the most fundamental and powerful algorithms in computer science, forming the basis for countless other algorithms.

2. Intuition

Simple explanation:

Look at the middle element of a sorted array. If it's what you want, done. If your target is smaller, search the left half. If larger, search the right half. Repeat until you find it or the interval is empty.

Analogy:

Think of looking up a word in a physical dictionary. You don't start from page 1. You open the dictionary somewhere in the middle. If the word comes alphabetically after the page you opened, you ignore the left half and look in the right half. Repeat until you find the word.

Step-by-step reasoning:

1. Look at the middle element of the current range.
2. If it matches the target — return its index.
3. If the target is smaller — search the left half.
4. If the target is larger — search the right half.
5. If the range is empty — target not found.

Why it works:

Each comparison eliminates half the remaining elements. Starting with n elements, after k comparisons, at most $n/2^k$ elements remain. Solve $n/2^k = 1 \rightarrow k = \log_2(n)$.

3. When to Use It

- When searching in a **sorted array**.
- When the **search space is monotonic** (a function that is consistently increasing/decreasing).
- When you need to find the **first/last occurrence** of a value.
- When you need to find the **peak** in a bitonic array.
- When you need to find the **minimum** in a rotated sorted array.
- When you're doing **predicate-based search** (find the boundary where a condition changes from true to false).
- When you need to **find an element in a sorted matrix**.
- When you're looking for **square root**, **cubic root**, or any monotonic function's root.

Common trigger phrases: "sorted array", "search", "find in $O(\log n)$ ", "first occurrence", "last occurrence", "binary search on answer", "minimize/maximize a value".

4. When Not to Use It

- When the array is **not sorted** — sort it first ($O(n \log n)$) or use linear search.
- For **small arrays** ($n < 10-20$) — linear search may be faster due to cache.
- When you need to **search frequently** in a **dynamic array** (insertions/deletions) — use a balanced BST or hash set.
- When the **comparison operation is expensive** — binary search still needs $O(\log n)$ comparisons.
- When the data is on **non-random-access storage** (linked list) — binary search is $O(n)$ since you need to traverse to the middle.

5. Core Concepts

Search Space

- The range of indices or values being searched.
- Usually defined by **low** and **high** boundaries.

Midpoint Calculation

- $mid = low + (high - low) / 2$ — avoids integer overflow.
- Never use $(low + high) / 2$ for large values (can overflow 32-bit int).

Three Templates

Template 1: Basic search (find exact value)

- `low = 0, high = n - 1`
- Loop while `low <= high`
- Return `mid` when found, else return -1.

Template 2: First/Last occurrence (lower/upper bound)

- `low = 0, high = n`
- Loop while `low < high`
- `mid = low + (high - low) / 2`
- `if (arr[mid] >= target) high = mid else low = mid + 1`
- This finds the **first** position where target could be inserted.

Template 3: Binary search on answer (predicate search)

- Used when the search space is a range of values, not indices.
- Define a predicate function `f(x)` that is monotonic (e.g., true for all $x \geq$ some threshold).
- Find the smallest/largest x where `f(x)` is true/false.

Predicate-based Search

- Instead of searching for a value, search for a **boundary**.
- Predicate: `can(arr, mid)` returns true/false.
- Find the point where the predicate changes.
- Used in: "minimum capacity to ship packages", "split array largest sum", "koko eating bananas".

6. Step-by-Step Algorithm

Standard Binary Search

```

binarySearch(arr, target):
  1. low = 0, high = len(arr) - 1
  2. while low <= high:
      mid = low + (high - low) / 2
      if arr[mid] == target: return mid
      if arr[mid] < target: low = mid + 1
      else: high = mid - 1
  3. return -1

```

Lower Bound (first occurrence / first $\geq$ target)

```

lowerBound(arr, target):
  1. low = 0, high = len(arr)
  2. while low < high:
      mid = low + (high - low) / 2
      if arr[mid] >= target: high = mid
      else: low = mid + 1
  3. return low

```

Upper Bound (first > target)

```
upperBound(arr, target):  
  1. low = 0, high = len(arr)  
  2. while low < high:  
      mid = low + (high - low) / 2  
      if arr[mid] > target: high = mid  
      else: low = mid + 1  
  3. return low
```

7. Dry Run

Standard Binary Search

Input: `arr = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]`, `target = 23`

Step	low	high	mid	arr[mid]	Action
1	0	9	4	16	16 < 23, go right
2	5	9	7	56	56 > 23, go left
3	5	6	5	23	Found! Return 5

Lower Bound

Input: `arr = [1, 3, 3, 3, 5, 7, 9]`, `target = 3`

Step	low	high	mid	arr[mid]	Action
1	0	7	3	3	arr[3] >= 3, high = 3
2	0	3	1	3	arr[1] >= 3, high = 1
3	0	1	0	1	arr[0] < 3, low = 1
4	1	1	loop ends	—	Return 1

8. C++ Implementation

```
#include <bits/stdc++.h>  
using namespace std;  
  
// Standard binary search – returns index or -1  
int binarySearch(const vector<int>& arr, int target) {  
    int low = 0, high = (int)arr.size() - 1;  
  
    while (low <= high) {  
        int mid = low + (high - low) / 2;  
  
        if (arr[mid] == target) return mid;  
    }  
}
```

```

        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

// First occurrence (lower bound)
int firstOccurrence(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size();
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] >= target) high = mid;
        else low = mid + 1;
    }
    return (low < (int)arr.size() && arr[low] == target) ? low : -1;
}

// Last occurrence
int lastOccurrence(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size();
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] > target) high = mid;
        else low = mid + 1;
    }
    // low is first > target, so last occurrence is low-1
    return (low - 1 >= 0 && arr[low - 1] == target) ? low - 1 : -1;
}

// Binary search on answer – predicate style
// Find the smallest value such that predicate(x) is true
int binarySearchOnAnswer(int low, int high, function<bool(int)> predicate) {
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (predicate(mid)) high = mid;
        else low = mid + 1;
    }
    return low;
}

// Example usage
int main() {
    vector<int> arr = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};

    cout << binarySearch(arr, 23) << "\n"; // 5
    cout << binarySearch(arr, 1) << "\n"; // -1

    vector<int> dup = {1, 3, 3, 3, 5, 7, 9};
    cout << firstOccurrence(dup, 3) << "\n"; // 1
    cout << lastOccurrence(dup, 3) << "\n"; // 3

    // Find smallest x such that x^2 >= 50
    auto pred = [](int x) { return x * x >= 50; };
    cout << binarySearchOnAnswer(1, 100, pred) << "\n"; // 8 (8^2=64 >= 50)
}

```

```
    return 0;
}
```

9. Python Implementation

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

def first_occurrence(arr, target):
    low, high = 0, len(arr)

    while low < high:
        mid = low + (high - low) // 2
        if arr[mid] >= target:
            high = mid
        else:
            low = mid + 1

    return low if low < len(arr) and arr[low] == target else -1

def last_occurrence(arr, target):
    low, high = 0, len(arr)

    while low < high:
        mid = low + (high - low) // 2
        if arr[mid] > target:
            high = mid
        else:
            low = mid + 1

    # low is first > target, so last occurrence is low - 1
    return low - 1 if low - 1 >= 0 and arr[low - 1] == target else -1

# Example usage
arr = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(binary_search(arr, 23)) # 5
```

```
print(binary_search(arr, 1))    # -1

dup = [1, 3, 3, 3, 5, 7, 9]
print(first_occurrence(dup, 3)) # 1
print(last_occurrence(dup, 3))  # 3
```

10. Code Explanation

Standard binary search:

- `low` and `high` define the current search range (inclusive).
- `mid = low + (high - low) / 2` avoids overflow.
- Three cases: equal (found), less (go right), greater (go left).
- Loop terminates when `low > high` — target not found.

Lower bound / first occurrence:

- Search range is `[0, n)` (high is exclusive).
- When `arr[mid] >= target`, move left (including mid).
- When `arr[mid] < target`, move right (excluding mid).
- Returns the first index where target can be inserted while maintaining order.

Binary search on answer:

- Search space is a range of values, not array indices.
- `predicate(mid)` returns true/false and must be monotonic.
- Finds the smallest value where predicate is true.

11. Complexity Analysis

Operation	Complexity
Time	$O(\log n)$
Space	$O(1)$ iterative, $O(\log n)$ recursive

12. Common Patterns

Pattern 1: Search in Rotated Sorted Array

- **How to identify:** Sorted array that was rotated at some pivot.
- **Approach:** Find which half is sorted (compare `arr[mid]` with `arr[low]`), then check if target lies in that sorted half.
- **Example:** Search in Rotated Sorted Array (LeetCode 33).

Pattern 2: Find Peak Element

- **How to identify:** "Find peak", "element greater than neighbors".
- **Approach:** Compare `arr[mid]` with `arr[mid+1]`. If `arr[mid] < arr[mid+1]`, peak is on the right. Otherwise, peak is on the left.

- **Example:** Find Peak Element (LeetCode 162).

Pattern 3: Binary Search on Answer

- **How to identify:** "Minimize maximum", "maximize minimum", "allocate resources", "capacity to ship".
- **Approach:** Binary search on the answer value. Define a predicate that checks if a given value is feasible.
- **Example:** Capacity to Ship Packages Within D Days (LeetCode 1011), Koko Eating Bananas (LeetCode 875).

Pattern 4: Search in 2D Matrix

- **How to identify:** Sorted matrix (row-wise and column-wise).
- **Approach:** Treat as 1D array: $mid = low + (high - low) / 2$, $row = mid / cols$, $col = mid \% cols$.
- **Example:** Search a 2D Matrix (LeetCode 74).

Pattern 5: Find Median of Two Sorted Arrays

- **How to identify:** "Median of two sorted arrays".
- **Approach:** Binary search on the smaller array to find the correct partition.
- **Example:** Median of Two Sorted Arrays (LeetCode 4).

13. Common Mistakes

- **Integer overflow in mid calculation:** $(low + high) / 2$ can overflow for large values. Use $low + (high - low) / 2$.
- **Infinite loop with $low < high$:** Without proper mid update, can loop forever. Use $mid = low + (high - low + 1) / 2$ for upper mid.
- **Off-by-one in boundary conditions:** Standard ($low \leq high$), lower bound ($low < high$), upper bound ($low < high$).
- **Wrong range:** For lower bound, $high = n$ (exclusive). For standard search, $high = n - 1$ (inclusive).
- **Not handling duplicates:** Standard binary search returns any occurrence. Use lower/upper bound for first/last.
- **Assuming sorted order:** Binary search only works on sorted arrays.
- **Not checking bounds:** After lower bound, check if $low < n$ and $arr[low] == target$ before returning.

14. Edge Cases

- Empty array: return -1.
- Single element: works correctly.
- Target at first position: $arr[0] == target$.
- Target at last position: $arr[n-1] == target$.
- Target not present: return -1 (or insertion point).
- Duplicate values: use lower/upper bound.
- All elements smaller than target: low goes to n.
- All elements larger than target: low stays at 0.
- Very large array ($n > 10^9$): binary search on answer still $O(\log n)$ with 64-bit ints.

15. Variations

Ternary Search

- Divides the search range into three parts.
- Used for unimodal functions (finding maximum/minimum).
- $O(\log_3 n)$ comparisons, but each step does 2 comparisons vs 1 for binary search.
- Usually slower in practice.

Exponential Search

- Starts with size 1, doubles until the range contains the target.
- Then performs binary search on that range.
- Useful for unbounded/infinite arrays.
- $O(\log n)$ time.

Interpolation Search

- Estimates the position based on value (like looking up in a phone book).
- $O(\log \log n)$ for uniformly distributed data.
- $O(n)$ worst-case.
- Rarely used in practice.

Binary Search Tree (BST)

- Tree-based data structure using binary search logic.
- Insert, delete, search in $O(\log n)$ average.
- Not the same as binary search on an array.

16. Related Algorithms

Algorithm	Difference	When to Choose
Linear Search	$O(n)$, no sorting needed	Small arrays, unsorted data
Ternary Search	$O(\log_3 n)$, 2 comparisons per step	Unimodal function optimization
Exponential Search	$O(\log n)$, works for unbounded arrays	When array size is unknown
Binary Search Tree	$O(\log n)$ avg, supports insert/delete	Dynamic data with frequent updates
Hash Set	$O(1)$ average, no ordering	Fast lookups, no need for range queries

17. Practice Problems

Easy

1. **Binary Search** — LeetCode 704 → Standard binary search implementation.
2. **First Bad Version** — LeetCode 278 → First occurrence pattern.

Medium

3. **Search in Rotated Sorted Array** — LeetCode 33 → Modified binary search.
4. **Find First and Last Position of Element in Sorted Array** — LeetCode 34 → Lower + upper bound.
5. **Koko Eating Bananas** — LeetCode 875 → Binary search on answer.

Hard

6. **Median of Two Sorted Arrays** — LeetCode 4 → Binary search on partitions.
7. **Split Array Largest Sum** — LeetCode 410 → Binary search on answer + greedy check.
8. **Kth Smallest Element in a Sorted Matrix** — LeetCode 378 → Binary search on value range.

18. Interview Explanation

"Binary Search is a divide-and-conquer algorithm for searching in a sorted array. It works by repeatedly comparing the target with the middle element and discarding the half that cannot contain the target. The key insight is that each comparison eliminates half the search space, giving $O(\log n)$ time. I use the standard template with `low = 0`, `high = n - 1` and `while (low <= high)`. For problems involving first or last occurrence, I switch to the lower bound template with `while (low < high)`. The most powerful extension is binary search on answer, where we binary search over the value range and use a feasibility predicate — this is extremely common in competitive programming and placement problems."

19. Revision Notes

- Only works on sorted data.
- `mid = low + (high - low) / 2` to avoid overflow.
- Three templates: exact value, lower/upper bound, predicate search.
- Lower bound: `if (arr[mid] >= target) high = mid else low = mid + 1.`
- Upper bound: `if (arr[mid] > target) high = mid else low = mid + 1.`
- Binary search on answer: define monotonic predicate, find boundary.
- Always check edge cases: empty, single, not found, duplicates.

20. Final Cheat Sheet

Aspect	Detail
When to use	Sorted array, monotonic search space, predicate-based search
Main operations	Compare mid, discard half
Complexity	Time: $O(\log n)$, Space: $O(1)$
Key code idea	<code>while (low <= high) { mid = low + (high-low)/2; ... }</code>
Edge cases	Empty, single, not found, duplicates, overflow
Common mistake	Off-by-one in high → use <code>low <= high</code> for exact, <code>low < high</code> for bound

Count Inversions

1. Overview

Counting inversions in an array measures how "unsorted" the array is. An **inversion** is a pair of indices (i, j) such that $i < j$ and $arr[i] > arr[j]$. The problem asks to count the total number of such pairs. The classic solution uses a modified Merge Sort to count in $O(n \log n)$ time.

2. Intuition

Simple explanation:

An inversion is any pair of elements that are out of order. Count how many pairs need to be swapped to sort the array.

Analogy:

Imagine a line of people where each person has a height number. An inversion is when a taller person is standing in front of a shorter person. Count how many such pairs exist.

Step-by-step reasoning:

1. Split the array into two halves.
2. Count inversions in the left half.
3. Count inversions in the right half.
4. Count inversions where one element is in the left half and the other is in the right half (cross inversions).
5. The key insight: when merging two sorted halves, if $arr[i] > arr[j]$ (where i is in left half, j in right half), then all remaining elements in the left half also form inversions with $arr[j]$.

Why it works:

During the merge step of Merge Sort, when we find that $L[i] > R[j]$, it means every element from $L[i]$ to the end of L is greater than $R[j]$ (since both halves are sorted). So we can count all those inversions at once, adding $(mid - i + 1)$ to the count.

3. When to Use It

- When a problem asks to **count pairs (i, j) where $i < j$ and $arr[i] > arr[j]$** .
- When a problem asks to measure **how sorted an array is**.
- When a problem asks for **"good pairs"** or **"bad pairs"** with a condition involving indices and values.
- When a problem involves **comparing elements across two halves**.
- When a problem asks for **"number of swaps needed to sort"** (minimum adjacent swaps = number of inversions).

Common trigger phrases: "count inversions", "number of pairs", " $i < j$ and $arr[i] > arr[j]$ ", "good pairs", "reverse pairs", "degree of unsortedness".

4. When Not to Use It

- When you need to **output the actual pairs** (count is $O(n \log n)$, enumerating all pairs is $O(n^2)$).
- When the array is **small** ($n \leq 1000$) — $O(n^2)$ brute force is simpler and acceptable.
- When the problem involves **non-adjacent swaps** or **minimum swaps to sort with arbitrary swaps** (that's a different problem using cycle detection).
- When the array contains **non-numeric elements** that can't be compared.

5. Core Concepts

Inversion Definition

- A pair (i, j) where $i < j$ and $arr[i] > arr[j]$.
- Maximum inversions: $n * (n - 1) / 2$ (reverse sorted array).
- Minimum inversions: 0 (already sorted array).

Cross Inversions

- Inversions where one element is in the left half and the other in the right half.
- During merge, when $L[i] > R[j]$, every element from $L[i]$ to the end of L is greater than $R[j]$ (since both halves are sorted).
- So we add $(L.size() - i)$ to the count.

Merge Sort Integration

- The inversion count is computed during the merge step.
- The sorting itself is still needed to maintain the invariant that subarrays are sorted.
- The algorithm returns the inversion count (not the sorted array, but we get both).

6. Step-by-Step Algorithm

```
countInversions(arr, left, right):
  1. If left >= right, return 0
  2. mid = (left + right) / 2
  3. inv = 0
  4. inv += countInversions(arr, left, mid)
  5. inv += countInversions(arr, mid + 1, right)
  6. inv += mergeAndCount(arr, left, mid, right)
  7. return inv

mergeAndCount(arr, left, mid, right):
  1. Create L = arr[left..mid], R = arr[mid+1..right]
  2. i = 0, j = 0, k = left, inv = 0
  3. While i < L.size() and j < R.size():
    if L[i] <= R[j]:
      arr[k++] = L[i++]
    else:
      arr[k++] = R[j++]
      inv += (L.size() - i) // All remaining L elements are > R[j]
  4. Copy remaining elements from L or R
  5. Return inv
```

7. Dry Run

Input: $[1, 3, 2, 4, 5]$

Split: [1, 3, 2] and [4, 5]

Left half [1, 3, 2]:

Split: [1, 3] and [2]

[1, 3]: split → [1] and [3] → merge → [1, 3], inv=0

Merge [1, 3] and [2]:

L=[1,3], R=[2]

i=0, j=0: L[0]=1 <= R[0]=2 → arr[0]=1, i=1

i=1, j=0: L[1]=3 > R[0]=2 → arr[1]=2, j=1, inv += (2-1) = 1

Copy remaining: arr[2]=3

inv = 1

Left half sorted: [1, 2, 3], inv=1

Right half [4, 5]:

Already sorted, inv=0

Merge [1, 2, 3] and [4, 5]:

L=[1,2,3], R=[4,5]

i=0, j=0: 1 <= 4 → arr[0]=1, i=1

i=1, j=0: 2 <= 4 → arr[1]=2, i=2

i=2, j=0: 3 <= 4 → arr[2]=3, i=3

Copy remaining: arr[3]=4, arr[4]=5

inv += 0

Total inversions: 1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long mergeAndCount(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    long long inv = 0;
    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
            // All remaining elements in L are > R[j]
            inv += (n1 - i);
        }
    }
}
```

```

    }

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    return inv;
}

long long countInversions(vector<int>& arr, int left, int right) {
    if (left >= right) return 0;

    int mid = left + (right - left) / 2;
    long long inv = 0;

    inv += countInversions(arr, left, mid);
    inv += countInversions(arr, mid + 1, right);
    inv += mergeAndCount(arr, left, mid, right);

    return inv;
}

// Example usage
int main() {
    vector<int> arr = {1, 3, 2, 4, 5};
    long long result = countInversions(arr, 0, arr.size() - 1);
    cout << "Inversions: " << result << "\n"; // 1
    return 0;
}

```

9. Python Implementation

```

def merge_and_count(arr, left, mid, right):
    L = arr[left:mid + 1]
    R = arr[mid + 1:right + 1]

    i = j = 0
    k = left
    inv = 0

    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
            inv += len(L) - i # All remaining elements in L are > R[j]
        k += 1

    while i < len(L):
        arr[k] = L[i]

```

```

        i += 1
        k += 1

    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1

    return inv

def count_inversions(arr, left, right):
    if left >= right:
        return 0

    mid = (left + right) // 2
    inv = 0

    inv += count_inversions(arr, left, mid)
    inv += count_inversions(arr, mid + 1, right)
    inv += merge_and_count(arr, left, mid, right)

    return inv

# Example usage
arr = [1, 3, 2, 4, 5]
result = count_inversions(arr, 0, len(arr) - 1)
print(f"Inversions: {result}") # 1

```

10. Code Explanation

mergeAndCount:

- Creates temporary arrays **L** and **R** for the two sorted halves.
- Standard merge sort comparison, but with an extra line:
 - When **L[i] > R[j]**, we add **(n1 - i)** to the inversion count.
 - This is because **L[i]** and all elements after it are greater than **R[j]** (since **L** is sorted).

countInversions:

- Recursively counts inversions in left half, right half, and cross inversions.
- Returns the total count.

Key detail:

- The array is sorted as a side effect — this is necessary for the algorithm to work.
- Use **long long** for the count because the maximum value is $n*(n-1)/2 \approx 5 \times 10^9$ for $n=10^5$, which fits in 32-bit signed int? No, 10^5 gives $\sim 5 \times 10^9$ which exceeds $2^{31}-1$ ($\approx 2.1 \times 10^9$). So use **long long**.

11. Complexity Analysis

Operation	Complexity
Time	$O(n \log n)$
Space	$O(n)$ auxiliary

12. Common Patterns

Pattern 1: Standard Inversion Count

- **How to identify:** "Count pairs (i, j) where $i < j$ and $arr[i] > arr[j]$ ".
- **Approach:** Merge Sort-based inversion count.
- **Example:** Count Inversions (GFG), Inversion Count (CSES).

Pattern 2: Reverse Pairs (LeetCode 493)

- **How to identify:** "Count pairs where $i < j$ and $arr[i] > 2 * arr[j]$ ".
- **Approach:** Same merge sort idea, but count during merge using a separate check (not during the actual merge).
- **Example:** Reverse Pairs (LeetCode 493).

Pattern 3: Count of Smaller Numbers After Self (LeetCode 315)

- **How to identify:** For each element, count how many elements to its right are smaller.
- **Approach:** Modified merge sort that tracks original indices. When merging, count elements from right half that are smaller than each element from left half.
- **Example:** Count of Smaller Numbers After Self (LeetCode 315).

Pattern 4: Global and Local Inversions (LeetCode 775)

- **How to identify:** Compare global inversions (any $i < j$) with local inversions (adjacent pairs).
- **Approach:** Local inversions = count where $arr[i] > arr[i+1]$. Global inversions = standard inversion count. If equal, return true.
- **Example:** Global and Local Inversions (LeetCode 775).

13. Common Mistakes

- **Using `int` for the count:** The maximum inversions is $n*(n-1)/2$. For $n = 10^5$, this is $\sim 5 \times 10^9$, which exceeds 32-bit int. Use `long long`.
- **Counting cross inversions incorrectly:** When $L[i] > R[j]$, add $(L.size() - i)$, not just 1.
- **Forgetting to sort the array:** The merge step must sort the array; otherwise, subarrays won't be sorted, and the count logic fails.
- **Double counting:** Make sure you're not counting the same inversion in multiple recursive calls.
- **Off-by-one in mid calculation:** Left half: $[left..mid]$, Right half: $[mid+1..right]$.

14. Edge Cases

- Empty array: 0 inversions.
- Single element: 0 inversions.
- Already sorted array: 0 inversions.

- Reverse sorted array: $n*(n-1)/2$ inversions.
- All equal elements: 0 inversions (since condition is $>$ not $\geq$).
- Large array ($n = 10^5$): should run in < 1 second with $O(n \log n)$.
- Array with negative values: works fine (comparison is still valid).

15. Variations

Count Inversions in a Range

- Count inversions where both elements are within a given index range.
- Can be solved with a segment tree / Fenwick tree for online queries.

Count Inversions with BIT/Fenwick Tree

- Alternative approach: process elements from left to right, use BIT to count elements greater than current.
- $O(n \log n)$ time, $O(n)$ space.
- Useful when the array values are large — coordinate compression needed.

Minimum Adjacent Swaps to Sort

- This is exactly the inversion count.
- Each swap fixes exactly one inversion.

16. Related Algorithms

Algorithm	Difference	When to Choose
Merge Sort	Base algorithm for $O(n \log n)$ inversion count	When you need both sorting and counting
Fenwick Tree (BIT)	Alternative $O(n \log n)$ approach	When you need online queries or per-element counts
Segment Tree	Similar to BIT, more flexible	When you need range queries with inversion-like logic
Brute Force	$O(n^2)$	Only for small arrays ($n \leq 1000$)

17. Practice Problems

Easy

1. **Count Inversions** — GFG → Standard inversion count.
2. **Minimum Adjacent Swaps to Sort** — Similar to inversion count.

Medium

3. **Reverse Pairs** — LeetCode 493 → Count pairs where $i < j$ and $arr[i] > 2*arr[j]$.
4. **Count of Smaller Numbers After Self** — LeetCode 315 → Per-element inversion count.
5. **Global and Local Inversions** — LeetCode 775 → Compare global vs local inversions.

Hard

- 6. **Count of Range Sum** — LeetCode 327 → Count range sums in a given range (merge sort approach).
- 7. **Number of Pairs Satisfying Inequality** — LeetCode 2426 → $\text{arr}[i] - \text{arr}[j] \leq \text{nums}[i] - \text{nums}[j] + k$.

18. Interview Explanation

"Counting inversions is a classic problem that can be solved in $O(n \log n)$ using a modified merge sort. The key insight is that during the merge step, when we find an element from the right half that is smaller than an element from the left half, all remaining elements in the left half are also larger — so we can count them all at once. The code is essentially merge sort with one extra line: `inv += (mid - i + 1)` when `arr[i] > arr[j]`. The total count is the sum of left inversions, right inversions, and cross inversions. This also solves the minimum adjacent swaps to sort problem."

19. Revision Notes

- Inversion: $i < j$ and `arr[i] > arr[j]`.
- Modified merge sort: add `inv += (L.size() - i)` when `L[i] > R[j]`.
- Use `long long` for count.
- Maximum inversions: $n*(n-1)/2$.
- Also solves "minimum adjacent swaps to sort".
- Alternative: Fenwick tree with coordinate compression.

20. Final Cheat Sheet

Aspect	Detail
When to use	Count pairs $(i < j)$ where <code>arr[i] > arr[j]</code>
Main operations	Merge sort + extra count during merge
Complexity	Time: $O(n \log n)$, Space: $O(n)$
Key code idea	<code>if (L[i] > R[j]) inv += (L.size() - i)</code>
Edge cases	Empty, single, sorted, reverse sorted, duplicates
Common mistake	Using <code>int</code> for count (overflow)

Maximum Subarray (Divide and Conquer)

1. Overview

The Maximum Subarray problem (also known as Kadane's Algorithm) asks: given an array of integers (which may include negative numbers), find the contiguous subarray with the largest sum. While Kadane's $O(n)$ algorithm is the standard solution, the divide-and-conquer approach provides a beautiful $O(n \log n)$ alternative that demonstrates the power of the divide-and-conquer paradigm.

2. Intuition

Simple explanation:

Find the contiguous subarray with the maximum sum. The divide-and-conquer approach splits the array, finds the best subarray in the left half, the best in the right half, and the best that crosses the middle — then returns the maximum of the three.

Analogy:

Imagine you're looking for the best stretch of road in a city. The best stretch could be entirely on the left side, entirely on the right side, or it could cross the bridge in the middle. Check all three and pick the best.

Step-by-step reasoning:

1. Split the array at the midpoint.
2. Recursively find the maximum subarray sum in the left half.
3. Recursively find the maximum subarray sum in the right half.
4. Find the maximum subarray sum that crosses the midpoint (extends into both halves).
5. Return the maximum of the three.

Why the cross subarray works:

The cross subarray must include some elements from the left half (ending at `mid`) and some from the right half (starting at `mid+1`). So we can compute:

- Best suffix sum ending at `mid` (going leftwards from `mid`).
- Best prefix sum starting at `mid+1` (going rightwards from `mid+1`).
- Add them together.

3. When to Use It

- When the problem asks for the **maximum subarray sum**.
- When the problem involves **contiguous subarrays** with some optimization criterion.
- When you're learning **divide-and-conquer** and want to see how recursion across the midpoint works.
- When the array is **large** and you need $O(n \log n)$ or better.
- When the problem is a **variation** that requires combining information from both halves (e.g., maximum subarray with constraints).

Common trigger phrases: "maximum subarray", "largest sum contiguous subarray", "maximum sum subarray", "best subarray".

4. When Not to Use It

- When you need the **optimal $O(n)$ solution** — Kadane's algorithm is simpler and faster.
- When the array is **small** — $O(n)$ Kadane's is trivial.
- When you need to solve the problem **quickly in an interview** — always start with Kadane's algorithm.
- When the problem involves **non-contiguous** subarrays (that's a different problem).

5. Core Concepts

Maximum Subarray Sum

- The maximum sum of any contiguous subarray.
- For `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`, the answer is `6` (subarray `[4, -1, 2, 1]`).

Cross Subarray (Mid-Crossing)

- A subarray that crosses the midpoint of the current range.
- Consists of a suffix of the left half + a prefix of the right half.
- Computed in $O(n)$ time by expanding outward from the midpoint.

Merge of Three Values

- The answer for a range is `max(leftBest, rightBest, crossBest)`.
- This is the "combine" step of divide-and-conquer.

6. Step-by-Step Algorithm

```

maxSubarray(arr, left, right):
  1. If left == right: return arr[left] (base case)
  2. mid = (left + right) / 2
  3. leftBest = maxSubarray(arr, left, mid)
  4. rightBest = maxSubarray(arr, mid + 1, right)
  5. crossBest = maxCrossing(arr, left, mid, right)
  6. return max(leftBest, rightBest, crossBest)

maxCrossing(arr, left, mid, right):
  1. // Find best suffix ending at mid
     sum = 0, leftSum = -INF
     for i = mid down to left:
       sum += arr[i]
       leftSum = max(leftSum, sum)

  2. // Find best prefix starting at mid+1
     sum = 0, rightSum = -INF
     for i = mid + 1 to right:
       sum += arr[i]
       rightSum = max(rightSum, sum)

  3. return leftSum + rightSum

```

7. Dry Run

Input: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`

```

Split: left=[-2, 1, -3, 4, -1], right=[2, 1, -5, 4]

Left half [-2, 1, -3, 4, -1]:
  Split: left=[-2, 1, -3], right=[4, -1]

  Left [-2, 1, -3]:
    Split: left=[-2, 1], right=[-3]
    Left [-2, 1]:
      Split: [-2], [1] → leftBest=-2, rightBest=1

```

Cross: max suffix ending at 0 = -2, max prefix starting at 1 = 1 → cross = -1

max(-2, 1, -1) = 1

Right [-3]: returns -3

Cross: best suffix ending at 2 = max(-3, -3+1, -3+1+-2) = -2? Let me redo this properly.

Actually, let's trace cross for [-2, 1, -3]:

mid = 2 (index of -3)

suffix ending at 2: -3, -3+1=-2, -3+1+(-2)=-4 → max = -2

prefix starting at 3: none → rightSum = 0 (or -INF)

cross = -2

max(1, -3, -2) = 1

Right [4, -1]: returns max(4, -1, 3) = 4

Cross: suffix ending at 3 = 4, suffix ending at 3 including 4+(-1) = 3 → max suffix = 4

prefix starting at 4 = -1 → max prefix = -1

cross = 4 + (-1) = 3

Cross for [-2, 1, -3, 4, -1]:

mid = 4 (index of -1)

suffix ending at 4: -1, -1+4=3, -1+4+(-3)=0, -1+4+(-3)+1=1, -1+4+(-3)+1+(-2)=-1 → max = 3

prefix starting at 5 (none) → rightSum = 0

cross = 3

max(1, 4, 3) = 4

Right half [2, 1, -5, 4]:

Split: left=[2, 1], right=[-5, 4]

Left [2, 1]: returns 3

Right [-5, 4]: returns 4

Cross: suffix ending at 6 = max(-5, -5+1=-4, -5+1+2=-2) = -2

prefix starting at 7 = max(4, 4+(-5)=-1) = 4

cross = -2 + 4 = 2

max(3, 4, 2) = 4

Cross for entire array:

mid = 4 (index of -1)

suffix ending at 4: -1, -1+4=3, -1+4+(-3)=0, -1+4+(-3)+1=1, -1+4+(-3)+1+(-2)=-1 → max = 3

prefix starting at 5: 2, 2+1=3, 2+1+(-5)=-2, 2+1+(-5)+4=2 → max = 3

cross = 3 + 3 = 6

Result: max(4, 4, 6) = 6

Subarray: [4, -1, 2, 1] with sum 6

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

int maxCrossing(const vector<int>& arr, int left, int mid, int right) {
    // Find maximum sum subarray ending at mid (going left)
    int leftSum = INT_MIN, sum = 0;
    for (int i = mid; i >= left; i--) {
        sum += arr[i];
        if (sum > leftSum) leftSum = sum;
    }

    // Find maximum sum subarray starting at mid+1 (going right)
    int rightSum = INT_MIN;
    sum = 0;
    for (int i = mid + 1; i <= right; i++) {
        sum += arr[i];
        if (sum > rightSum) rightSum = sum;
    }

    return leftSum + rightSum;
}

int maxSubarraySum(const vector<int>& arr, int left, int right) {
    if (left == right) return arr[left];

    int mid = left + (right - left) / 2;

    int leftBest = maxSubarraySum(arr, left, mid);
    int rightBest = maxSubarraySum(arr, mid + 1, right);
    int crossBest = maxCrossing(arr, left, mid, right);

    return max({leftBest, rightBest, crossBest});
}

// Example usage
int main() {
    vector<int> arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    int result = maxSubarraySum(arr, 0, arr.size() - 1);
    cout << "Maximum subarray sum: " << result << "\n"; // 6
    return 0;
}

```

9. Python Implementation

```

import sys

def max_crossing(arr, left, mid, right):
    # Best suffix ending at mid
    left_sum = -sys.maxsize - 1

```

```

    s = 0
    for i in range(mid, left - 1, -1):
        s += arr[i]
        left_sum = max(left_sum, s)

    # Best prefix starting at mid + 1
    right_sum = -sys.maxsize - 1
    s = 0
    for i in range(mid + 1, right + 1):
        s += arr[i]
        right_sum = max(right_sum, s)

    return left_sum + right_sum

def max_subarray_sum(arr, left, right):
    if left == right:
        return arr[left]

    mid = (left + right) // 2

    left_best = max_subarray_sum(arr, left, mid)
    right_best = max_subarray_sum(arr, mid + 1, right)
    cross_best = max_crossing(arr, left, mid, right)

    return max(left_best, right_best, cross_best)

# Example usage
arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
result = max_subarray_sum(arr, 0, len(arr) - 1)
print(f"Maximum subarray sum: {result}") # 6

```

10. Code Explanation

maxCrossing:

- Computes the maximum subarray that crosses the midpoint.
- Left part: iterate from `mid` down to `left`, calculating running sum. Track the maximum sum seen — this is the best suffix of the left half.
- Right part: iterate from `mid+1` to `right`, calculating running sum. Track the maximum sum seen — this is the best prefix of the right half.
- Return `leftSum + rightSum`.

maxSubarraySum:

- Base case: single element, return it.
- Recursively compute best in left, best in right, and best crossing.
- Return the maximum of the three.

Key detail: The cross sum is guaranteed to be contiguous because the suffix from left and prefix from right meet at the midpoint.

11. Complexity Analysis

Operation	Complexity
Time	$O(n \log n)$
Space	$O(\log n)$ recursion stack

Compared to Kadane's $O(n)$: This is slower, but it's a good demonstration of divide-and-conquer.

12. Common Patterns

Pattern 1: Maximum Subarray with Kadane's Algorithm

- **How to identify:** "Maximum subarray sum" — interviewer expects $O(n)$.
- **Approach:** Keep running sum, reset to 0 if negative.
- **Example:** Maximum Subarray (LeetCode 53).

Pattern 2: Maximum Subarray with Constraints

- **How to identify:** "Maximum subarray with at most k elements", "maximum subarray with one deletion".
- **Approach:** Divide and conquer with additional state (like best prefix/suffix for each half).
- **Example:** Maximum Subarray Sum with One Deletion (LeetCode 1186).

Pattern 3: Maximum Sum Circular Subarray

- **How to identify:** "Circular array", "maximum sum subarray in a circular array".
- **Approach:** Use Kadane's for normal max, and total - min subarray sum for wrap-around.
- **Example:** Maximum Sum Circular Subarray (LeetCode 918).

13. Common Mistakes

- **Not handling all negative numbers:** The algorithm should return the largest element (least negative). The base case handles this.
- **INT_MIN initialization:** Must initialize to `INT_MIN` (or `-inf`) not 0, because if all numbers are negative, the max sum is negative.
- **Off-by-one in cross computation:** The suffix must include elements from `mid` down to `left`. The prefix must include elements from `mid+1` to `right`.
- **Stack overflow for large arrays:** The recursion depth is $O(\log n)$, so this is fine for n up to 10^6 .

14. Edge Cases

- Empty array: should handle separately (return 0 or `INT_MIN` depending on definition).
- Single element: return that element.
- All negative numbers: return the largest (least negative) element.
- All positive numbers: return the sum of the entire array.

- Mixed positive and negative numbers: standard behavior.
- Large negative numbers: ensure `INT_MIN` is used correctly.

15. Variations

Kadane's Algorithm ($O(n)$)

- Iterative DP: `maxEndingHere = max(arr[i], maxEndingHere + arr[i]), maxSoFar = max(maxSoFar, maxEndingHere)`.
- This is the standard solution for the maximum subarray problem.
- $O(n)$ time, $O(1)$ space.

Maximum Subarray with One Deletion

- Use DP with two states: with and without deletion.
- Can also be solved with divide and conquer by tracking additional information.

Maximum Subarray in 2D

- Given a 2D matrix, find the submatrix with maximum sum.
- Can be solved by iterating over row pairs and using Kadane's on the column sums.

16. Related Algorithms

Algorithm	Difference	When to Choose
Kadane's Algorithm	$O(n)$, iterative DP	Always prefer this for max subarray
Divide & Conquer	$O(n \log n)$, recursive	Learning tool, or when extending to other constraints
Segment Tree	Can answer max subarray for any range	When you need range queries on dynamic arrays

17. Practice Problems

Easy

1. **Maximum Subarray** — LeetCode 53 → Kadane's algorithm (know both Kadane and D&C).
2. **Best Time to Buy and Sell Stock** — LeetCode 121 → Can be viewed as max subarray of differences.

Medium

3. **Maximum Sum Circular Subarray** — LeetCode 918 → Circular version.
4. **Maximum Subarray Sum with One Deletion** — LeetCode 1186 → Allow one deletion.

Hard

5. **Maximum Sum of 3 Non-Overlapping Subarrays** — LeetCode 689 → DP + prefix/suffix.
6. **K-Concatenation Maximum Sum** — LeetCode 1191 → Kadane's on concatenated arrays.

18. Interview Explanation

"The maximum subarray problem asks for the contiguous subarray with the largest sum. While Kadane's algorithm is the optimal $O(n)$ solution, the divide-and-conquer approach is also important to understand. It splits the array at the midpoint, recursively finds the best subarray in the left and right halves, and then computes the best subarray crossing the midpoint by combining the best suffix of the left half with the best prefix of the right half. The answer is the maximum of the three. This runs in $O(n \log n)$ time with $O(\log n)$ recursion depth. In practice, I'd use Kadane's algorithm for the $O(n)$ solution, but the divide-and-conquer approach is valuable for understanding the pattern and for variations like segment tree queries."

19. Revision Notes

- Divide: split at mid, Conquer: find best in left, right, Combine: max of three.
- Cross sum = best suffix ending at mid + best prefix starting at mid+1.
- Base case: single element.
- Alternative: Kadane's algorithm ($O(n)$, $O(1)$).
- All negative numbers: algorithm returns the largest element.
- Use `INT_MIN` for initialization.

20. Final Cheat Sheet

Aspect	Detail
When to use	Learning D&C, problems requiring cross-half information
Main operations	Left best + right best + cross best
Complexity	Time: $O(n \log n)$, Space: $O(\log n)$
Key code idea	<code>max(leftBest, rightBest, crossBest)</code>
Edge cases	All negative, single element, empty
Common mistake	Using 0 instead of <code>INT_MIN</code> for initialization

Closest Pair of Points

1. Overview

The Closest Pair of Points problem asks: given n points in a 2D plane, find the pair of points with the smallest Euclidean distance between them. The naive $O(n^2)$ solution checks every pair. The divide-and-conquer solution achieves $O(n \log n)$ time and is a classic example of the "combine" step in divide-and-conquer algorithms.

2. Intuition

Simple explanation:

Sort points by x-coordinate, split into two halves, recursively find the closest pair in each half, then check only a limited number of points near the dividing line that could be closer than the current best.

Analogy:

Imagine you have a map with many pins. To find the closest pair, you can split the map with a vertical line. The closest pair is either both on the left, both on the right, or they straddle the line. For the straddling case, you only need to check points within a narrow strip near the line — and for each point, you only need to check a few points below it.

Step-by-step reasoning:

1. Sort points by x-coordinate.
2. Split into left and right halves by the median x.
3. Recursively find the minimum distance d in each half.
4. Let $d = \min(d_{\text{left}}, d_{\text{right}})$.
5. Consider only points within d distance of the dividing line (the "strip").
6. Sort strip points by y-coordinate.
7. For each point in the strip, check only the next 7 points (or up to 15 in theory) — those within d distance in y.
8. Update d if a closer pair is found.

Why checking only 7 points works:

Within a $d \times d$ square, no more than 4 points can exist (since the minimum distance in each half is at least d). So for each point, only a constant number of points need to be checked.

3. When to Use It

- When finding the **minimum distance between any two points** in a plane.
- When finding the **closest pair of points** in 2D or 3D.
- When the **naive $O(n^2)$ is too slow** ($n > 10^4$).
- When solving problems involving **point sets** and **distance minimization**.

Common trigger phrases: "closest pair", "minimum distance between points", "find the pair with smallest distance", "nearest neighbor".

4. When Not to Use It

- For **1D points** — just sort and check consecutive differences ($O(n \log n)$, much simpler).
- For **small n** (≤ 1000) — $O(n^2)$ is simpler and acceptable.
- When you need **all pairs within a certain distance** — that's a range query problem.
- When points are **static** and you need **multiple queries** — use a spatial data structure (KD-tree, Voronoi diagram).
- When the **distance metric is not Euclidean** (e.g., Manhattan) — the algorithm still works, but the strip width and point count analysis may differ.

5. Core Concepts

Euclidean Distance

- $\text{distance}(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- Often we compare squared distances to avoid floating-point issues.

Divide by X-Median

- Sort by x-coordinate.
- Split at the median to ensure balanced halves.
- This gives $O(\log n)$ recursion depth.

The Strip

- A vertical band of width $2d$ centered at the dividing line.
- Only points in this strip can form a pair closer than d across the halves.
- Width is $2d$ because if a point is outside this strip, its distance to any point on the other side is at least d .

Y-Sorted Strip

- Sort the strip points by y-coordinate.
- For each point, only check the next 7-15 points (those within d in y-distance).
- This is the key to the $O(n \log n)$ complexity.

6. Step-by-Step Algorithm

```
closestPair(points):
    1. Sort points by x-coordinate.
    2. Return closestPairRecursive(points, 0, n-1)

closestPairRecursive(points, left, right):
    1. If right - left <= 3:
        Brute-force compute minimum distance ( $O(1)$  or  $O(n^2)$  for small cases)
    2. mid = (left + right) / 2
    3. midX = points[mid].x
    4. dLeft = closestPairRecursive(points, left, mid)
    5. dRight = closestPairRecursive(points, mid+1, right)
    6. d = min(dLeft, dRight)

    7. // Build strip: points with |x - midX| < d
       Create strip vector
    8. Sort strip by y-coordinate

    9. For i = 0 to strip.size() - 1:
        For j = i+1 to strip.size() - 1:
            If strip[j].y - strip[i].y >= d: break
            d = min(d, distance(strip[i], strip[j]))

    10. Return d
```

7. Dry Run

Input: `points = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]`

Sorted by x: `[(2,3), (3,4), (5,1), (12,10), (12,30), (40,50)]`

```

Step 1: Split at mid = 2 (index 2, x=5)
Left: [(2,3), (3,4), (5,1)]
Right: [(12,10), (12,30), (40,50)]

Step 2: Left half [(2,3), (3,4), (5,1)]
n=3, brute force:
  dist((2,3), (3,4)) = sqrt(2) ≈ 1.414
  dist((2,3), (5,1)) = sqrt(32+22) = sqrt(13) ≈ 3.606
  dist((3,4), (5,1)) = sqrt(22+32) = sqrt(13) ≈ 3.606
dLeft = 1.414

Step 3: Right half [(12,10), (12,30), (40,50)]
n=3, brute force:
  dist((12,10), (12,30)) = 20
  dist((12,10), (40,50)) = sqrt(282+402) = sqrt(2384) ≈ 48.83
  dist((12,30), (40,50)) = sqrt(282+202) = sqrt(1184) ≈ 34.41
dRight = 20

Step 4: d = min(1.414, 20) = 1.414
midX = 5

Step 5: Build strip (points with |x - 5| < 1.414)
Points with x in [3.586, 6.414]:
  (2,3) - |2-5|=3 > 1.414, skip
  (3,4) - |3-5|=2 > 1.414, skip
  (5,1) - |5-5|=0 < 1.414, include
  (12,10) - |12-5|=7 > 1.414, skip
Strip = [(5,1)]

Only one point in strip, no cross pairs to check.

Final answer: d = 1.414 (pair (2,3) and (3,4))

```

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct Point {
    double x, y;
};

double distance(const Point& a, const Point& b) {
    double dx = a.x - b.x, dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

// Brute force for small sets
double bruteForce(const vector<Point>& points, int left, int right) {
    double minDist = DBL_MAX;
    for (int i = left; i <= right; i++) {

```

```

        for (int j = i + 1; j <= right; j++) {
            minDist = min(minDist, distance(points[i], points[j]));
        }
    }
    return minDist;
}

double closestPairRecursive(vector<Point>& pointsX, int left, int right) {
    if (right - left <= 3) {
        return bruteForce(pointsX, left, right);
    }

    int mid = left + (right - left) / 2;
    double midX = pointsX[mid].x;

    double dLeft = closestPairRecursive(pointsX, left, mid);
    double dRight = closestPairRecursive(pointsX, mid + 1, right);
    double d = min(dLeft, dRight);

    // Build strip
    vector<Point> strip;
    for (int i = left; i <= right; i++) {
        if (abs(pointsX[i].x - midX) < d) {
            strip.push_back(pointsX[i]);
        }
    }

    // Sort strip by y
    sort(strip.begin(), strip.end(), [](const Point& a, const Point& b) {
        return a.y < b.y;
    });

    // Check points in strip
    for (size_t i = 0; i < strip.size(); i++) {
        for (size_t j = i + 1; j < strip.size() && (strip[j].y - strip[i].y) < d;
j++) {
            d = min(d, distance(strip[i], strip[j]));
        }
    }

    return d;
}

double closestPair(vector<Point>& points) {
    // Sort by x-coordinate
    sort(points.begin(), points.end(), [](const Point& a, const Point& b) {
        return a.x < b.x;
    });
    return closestPairRecursive(points, 0, points.size() - 1);
}

// Example usage
int main() {
    vector<Point> points = {{2, 3}, {12, 30}, {40, 50}, {5, 1}, {12, 10}, {3, 4}};

```

```

double result = closestPair(points);
cout << "Closest distance: " << result << "\n"; // ~1.414
return 0;
}

```

9. Python Implementation

```

import math

def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)

def brute_force(points, left, right):
    min_dist = float('inf')
    for i in range(left, right + 1):
        for j in range(i + 1, right + 1):
            min_dist = min(min_dist, distance(points[i], points[j]))
    return min_dist

def closest_pair_recursive(points_x, left, right):
    if right - left <= 3:
        return brute_force(points_x, left, right)

    mid = (left + right) // 2
    mid_x = points_x[mid][0]

    d_left = closest_pair_recursive(points_x, left, mid)
    d_right = closest_pair_recursive(points_x, mid + 1, right)
    d = min(d_left, d_right)

    # Build strip
    strip = [points_x[i] for i in range(left, right + 1)
              if abs(points_x[i][0] - mid_x) < d]

    # Sort strip by y
    strip.sort(key=lambda p: p[1])

    # Check points in strip
    for i in range(len(strip)):
        for j in range(i + 1, len(strip)):
            if strip[j][1] - strip[i][1] >= d:
                break
            d = min(d, distance(strip[i], strip[j]))

    return d

def closest_pair(points):

```

```

points_x = sorted(points, key=lambda p: p[0])
return closest_pair_recursive(points_x, 0, len(points_x) - 1)

# Example usage
points = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
result = closest_pair(points)
print(f"Closest distance: {result:.4f}") # ~1.4142

```

10. Code Explanation

closestPairRecursive:

- Base case: if 3 or fewer points, use brute force ($O(1)$).
- Recursively find d in left and right halves.
- Build the strip: only points within d of the dividing line.
- Sort strip by y (this is $O(k \log k)$ where k is the number of strip points).
- For each point in strip, check the next points until the y -difference exceeds d .

Key optimizations:

- The inner loop breaks early when $\text{strip}[j].y - \text{strip}[i].y \geq d$ because points are sorted by y .
- Only 7-15 points on average are checked per point.

Complexity note: The sorting of strip by y at each recursion level adds $O(n \log^2 n)$ if done naively. An optimized version pre-sorts by y and passes the sorted list, achieving $O(n \log n)$.

11. Complexity Analysis

Operation	Complexity
Time (basic)	$O(n \log^2 n)$
Time (optimized, pre-sort y)	$O(n \log n)$
Space	$O(n)$
Naive brute force	$O(n^2)$

12. Common Patterns

Pattern 1: Closest Pair in 1D

- **How to identify:** Points on a line.
- **Approach:** Sort, check consecutive differences.
- **Example:** Minimum distance between two elements in an array.

Pattern 2: Closest Pair in 3D

- **How to identify:** Points in 3D space.
- **Approach:** Same algorithm extended to 3D (strip becomes a volume).

- **Note:** More complex, but same idea.

Pattern 3: K Closest Pairs

- **How to identify:** Find the k closest pairs.
- **Approach:** Use a priority queue + closest pair algorithm.
- **Example:** Find K Closest Pairs.

13. Common Mistakes

- **Not handling floating-point precision:** Use epsilon comparisons when needed.
- **Wrong strip width:** The strip should be $[midX - d, midX + d]$, not $[midX - d, midX + d]$.
- **Not sorting strip by y:** The algorithm only works if strip points are sorted by y for the inner loop.
- **Incorrect inner loop bound:** The inner loop breaks when $y_j - y_i \geq d$, not when $j > i + 7$.
- **Not using squared distance for comparison:** To avoid sqrt, compare squared distances in the inner loop, then take sqrt at the end.
- **Stack overflow for large n:** Recursion depth is $O(\log n)$, so fine for n up to 10^6 .

14. Edge Cases

- Less than 2 points: return INF (no pair exists).
- 2 points: return their distance.
- All points with same x-coordinate: the algorithm degrades but still works (strip contains all points).
- All points with same y-coordinate: works fine.
- All points collinear: works fine.
- Points with very large coordinates: use double precision.
- Integer coordinates: use squared distances for comparison.

15. Variations

Closest Pair using Sweep Line

- Alternative $O(n \log n)$ approach.
- Process points sorted by y, maintain a balanced BST of points within d in x.
- Similar complexity, different approach.

Closest Pair in Higher Dimensions

- The same algorithm extends to 3D and beyond.
- The "strip" becomes a volume, and the constant factor grows exponentially with dimensions.
- Usually, KD-tree or R-tree is used for higher dimensions.

All Nearest Neighbors

- For each point, find its nearest neighbor.
- Can be solved with a similar D&C approach or using a Voronoi diagram.

16. Related Algorithms

Algorithm	Difference	When to Choose
Brute Force	$O(n^2)$, trivial	Small n (≤ 1000)
Divide & Conquer	$O(n \log n)$	Large n , 2D points
KD-Tree	$O(\log n)$ average per query	Multiple queries, dynamic points
Voronoi Diagram	$O(n \log n)$ for construction	Nearest neighbor queries, spatial analysis
Sweep Line	$O(n \log n)$	When points can be sorted by both coordinates

17. Practice Problems

Easy

1. **Minimum Distance Between Two Numbers** — GFG → 1D version.
2. **Closest Pair of Points (Brute Force)** — basic implementation.

Medium

3. **Closest Pair of Points** — GFG / Codeforces → Standard 2D closest pair.
4. **Find the Minimum Distance Between Two Points** — LeetCode-style → Practice implementation.

Hard

5. **K Closest Points to Origin** — LeetCode 973 → Different problem (uses heap), but related.
6. **Closest Pair of Points (Divide and Conquer Implementation)** — SPOJ → Full implementation required.

18. Interview Explanation

"The Closest Pair of Points problem finds the minimum Euclidean distance between any two points in a 2D plane. The naive $O(n^2)$ solution checks all pairs. The divide-and-conquer solution runs in $O(n \log n)$. We sort points by x , split at the median, recursively find the minimum distance in each half, and let d be the minimum of the two. The key insight is that any closer pair must straddle the dividing line, and such points must lie within a vertical strip of width $2d$. We sort the strip points by y , and for each point, we only need to check a constant number of points below it because within a $d \times d$ square, no more than 4 points can exist. This gives $O(n \log n)$ overall."

19. Revision Notes

- Sort by x , split at median, recurse.
- $d = \min(d_{\text{left}}, d_{\text{right}})$.
- Strip: points with $|x - \text{midX}| < d$.
- Sort strip by y , check each point against next points until y difference $\geq d$.
- Only 7-15 points need to be checked per point.
- Base case: $n \leq 3 \rightarrow$ brute force.
- Use squared distances to avoid sqrt until the end.

20. Final Cheat Sheet

Aspect	Detail
When to use	Find minimum distance between any two 2D points
Main operations	Recursive split, strip building, y-sort, constant-time pair check
Complexity	Time: $O(n \log n)$, Space: $O(n)$
Key code idea	Strip with
Edge cases	< 2 points, all same x, all same y, collinear points
Common mistake	Not sorting strip by y, wrong strip width

Karatsuba Multiplication

1. Overview

Karatsuba Multiplication is a **divide-and-conquer** algorithm for multiplying two large integers that is faster than the traditional grade-school multiplication. While grade-school multiplication is $O(n^2)$, Karatsuba achieves $O(n^{\log_2 3}) \approx O(n^{1.585})$, making it significantly faster for large numbers.

2. Intuition

Simple explanation:

To multiply two large numbers, split each into two halves, compute three products instead of four, and combine them using addition and subtraction.

Analogy:

Traditional multiplication of two n -digit numbers requires 4 multiplications of $n/2$ -digit numbers (via the formula $(a \cdot 10^k + b)(c \cdot 10^k + d) = ac \cdot 10^{2k} + (ad + bc) \cdot 10^k + bd$). Karatsuba observed that $ad + bc = (a+b)(c+d) - ac - bd$, which reuses the ac and bd products. So we only need 3 multiplications instead of 4.

Step-by-step reasoning:

1. Split the numbers into high and low halves.
2. Recursively compute:
 - $p1 = a * c$ (high $\times$ high)
 - $p2 = b * d$ (low $\times$ low)
 - $p3 = (a+b) * (c+d)$ (sum $\times$ sum)
3. Compute $p4 = p3 - p1 - p2$ (which equals $a*d + b*c$).
4. Result = $p1 * 10^{(2k)} + p4 * 10^k + p2$.

Why it works:

The formula $(a \cdot 10^k + b)(c \cdot 10^k + d) = ac \cdot 10^{2k} + (ad + bc) \cdot 10^k + bd$ is exact. By computing $(a+b)(c+d) - ac - bd$, we get $ad + bc$ with one extra multiplication instead of two.

3. When to Use It

- When multiplying **very large integers** (hundreds or thousands of digits).
- When implementing **big integer arithmetic** in languages without built-in support.
- When you need to demonstrate **divide-and-conquer on a non-trivial problem**.
- When the numbers are so large that $O(n^2)$ is too slow ($n > 1000$ digits).

Common trigger phrases: "multiply large numbers", "big integer multiplication", "fast multiplication", "Karatsuba".

4. When Not to Use It

- For **small numbers** (fits in native int/long) — just use the `*` operator.
- When the numbers are **moderately large** ($n < 100$ digits) — the overhead of Karatsuba may not be worth it.
- When you need to multiply **floating-point numbers** — different algorithms apply.
- When the numbers are **not in base 10** (but the algorithm works for any base).
- When **Toom-Cook or FFT-based multiplication** is even faster (for extremely large numbers).

5. Core Concepts

Split into High and Low Parts

- For an n -digit number x , split into a (high $n/2$ digits) and b (low $n/2$ digits).
- $x = a * 10^k + b$ where $k = n/2$.

The Three Products

- $p1 = a * c$ — high part product.
- $p2 = b * d$ — low part product.
- $p3 = (a+b) * (c+d)$ — sum product.

The Combine Formula

- $ad + bc = p3 - p1 - p2$.
- Result = $p1 * 10^{(2k)} + (p3 - p1 - p2) * 10^k + p2$.

Base Case

- When numbers are small (e.g., 1-2 digits), use standard multiplication.
- Typically, when $n < 10$ or $n < 16$, use `operator*` directly.

6. Step-by-Step Algorithm

```
karatsuba(x, y):
    1. If  $x < 10$  or  $y < 10$ : return  $x * y$  (base case)
    2.  $n = \max(\text{number of digits in } x, \text{number of digits in } y)$ 
    3.  $k = n / 2$ 
    4. Split  $x$  into  $a$  (high) and  $b$  (low):  $x = a * 10^k + b$ 
    5. Split  $y$  into  $c$  (high) and  $d$  (low):  $y = c * 10^k + d$ 
    6.  $p1 = \text{karatsuba}(a, c)$ 
    7.  $p2 = \text{karatsuba}(b, d)$ 
```

```

8. p3 = karatsuba(a + b, c + d)
9. return p1 * 10^(2*k) + (p3 - p1 - p2) * 10^k + p2

```

7. Dry Run

Input: $x = 1234$, $y = 5678$

```

n = 4, k = 2
a = 12, b = 34
c = 56, d = 78

p1 = karatsuba(12, 56)
  n = 2, k = 1
  a = 1, b = 2
  c = 5, d = 6
  p1 = 1 * 5 = 5
  p2 = 2 * 6 = 12
  p3 = karatsuba(1+2, 5+6) = karatsuba(3, 11) = 33
  result = 5 * 10^2 + (33 - 5 - 12) * 10^1 + 12
           = 500 + 16 * 10 + 12
           = 500 + 160 + 12 = 672
  p1 = 672

p2 = karatsuba(34, 78)
  n = 2, k = 1
  a = 3, b = 4
  c = 7, d = 8
  p1 = 3 * 7 = 21
  p2 = 4 * 8 = 32
  p3 = karatsuba(3+4, 7+8) = karatsuba(7, 15) = 105
  result = 21 * 100 + (105 - 21 - 32) * 10 + 32
           = 2100 + 52 * 10 + 32
           = 2100 + 520 + 32 = 2652
  p2 = 2652

p3 = karatsuba(12+34, 56+78) = karatsuba(46, 134)
  n = 3, k = 1
  a = 4, b = 6
  c = 13, d = 4
  Wait, splitting 46 into a=4, b=6 (k=1)
  Splitting 134 into c=13, d=4

  p1 = karatsuba(4, 13) = 52
  p2 = karatsuba(6, 4) = 24
  p3 = karatsuba(4+6, 13+4) = karatsuba(10, 17) = 170
  result = 52 * 10^2 + (170 - 52 - 24) * 10^1 + 24
           = 5200 + 94 * 10 + 24
           = 5200 + 940 + 24 = 6164
  p3 = 6164

```

Final result:

$$\begin{aligned}
 & p1 * 10^4 + (p3 - p1 - p2) * 10^2 + p2 \\
 &= 672 * 10000 + (6164 - 672 - 2652) * 100 + 2652 \\
 &= 6,720,000 + (2840) * 100 + 2652 \\
 &= 6,720,000 + 284,000 + 2652 \\
 &= 7,006,652
 \end{aligned}$$

Check: $1234 * 5678 = 7,006,652 \checkmark$

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// Helper: make two numbers have the same length by padding with leading zeros
string addStrings(const string& a, const string& b) {
    string result;
    int i = a.size() - 1, j = b.size() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry;
        if (i >= 0) sum += a[i--] - '0';
        if (j >= 0) sum += b[j--] - '0';
        carry = sum / 10;
        result.push_back('0' + (sum % 10));
    }
    reverse(result.begin(), result.end());
    return result;
}

string subtractStrings(const string& a, const string& b) {
    // Assumes a >= b
    string result;
    int i = a.size() - 1, j = b.size() - 1, borrow = 0;
    while (i >= 0) {
        int diff = (a[i--] - '0') - borrow;
        if (j >= 0) diff -= (b[j--] - '0');
        if (diff < 0) { diff += 10; borrow = 1; }
        else borrow = 0;
        result.push_back('0' + diff);
    }
    while (result.size() > 1 && result.back() == '0') result.pop_back();
    reverse(result.begin(), result.end());
    return result;
}

string multiplyByPower10(const string& s, int k) {
    if (s == "0") return s;
    return s + string(k, '0');
}

```

```

string karatsuba(const string& x, const string& y) {
    // Base case: small numbers
    if (x.size() <= 4 || y.size() <= 4) {
        ll a = stoll(x), b = stoll(y);
        return to_string(a * b);
    }

    // Make lengths equal
    int n = max(x.size(), y.size());
    string X = string(n - x.size(), '0') + x;
    string Y = string(n - y.size(), '0') + y;

    int k = n / 2;
    string a = X.substr(0, n - k); // high part
    string b = X.substr(n - k);    // low part
    string c = Y.substr(0, n - k);
    string d = Y.substr(n - k);

    // Remove leading zeros
    auto trim = [](string& s) {
        int pos = 0;
        while (pos + 1 < (int)s.size() && s[pos] == '0') pos++;
        s = s.substr(pos);
    };
    trim(a); trim(b); trim(c); trim(d);

    string p1 = karatsuba(a, c);
    string p2 = karatsuba(b, d);
    string p3 = karatsuba(addStrings(a, b), addStrings(c, d));

    // (p3 - p1 - p2)
    string p4 = subtractStrings(subtractStrings(p3, p1), p2);

    // Result: p1 * 10^(2k) + p4 * 10^k + p2
    string result = addStrings(addStrings(multiplyByPower10(p1, 2 * k),
                                           multiplyByPower10(p4, k)),
                             p2);

    // Remove leading zeros
    trim(result);
    return result;
}

// Example usage
int main() {
    string x = "1234", y = "5678";
    string result = karatsuba(x, y);
    cout << x << " * " << y << " = " << result << "\n"; // 7006652
    return 0;
}

```

9. Python Implementation

```
def karatsuba(x, y):
    # Base case: small numbers
    if x < 10 or y < 10:
        return x * y

    # Make lengths equal
    n = max(len(str(x)), len(str(y)))
    k = n // 2

    # Split into high and low parts
    a = x // (10 ** k)
    b = x % (10 ** k)
    c = y // (10 ** k)
    d = y % (10 ** k)

    # Three recursive multiplications
    p1 = karatsuba(a, c)
    p2 = karatsuba(b, d)
    p3 = karatsuba(a + b, c + d)

    # Combine:  $p1 * 10^{(2k)} + (p3 - p1 - p2) * 10^k + p2$ 
    return p1 * (10 ** (2 * k)) + (p3 - p1 - p2) * (10 ** k) + p2

# Example usage
x = 1234
y = 5678
result = karatsuba(x, y)
print(f"{x} * {y} = {result}") # 7006652
```

10. Code Explanation

Base case:

- When numbers are small enough, use standard multiplication.
- The C++ version uses strings for arbitrary precision, so the base case handles numbers up to 4 digits.

Splitting:

- n is the maximum digit count of both numbers.
- $k = n/2$ determines the split point.
- a and c are the high parts (most significant digits).
- b and d are the low parts (least significant digits).

Three recursive calls:

- $p1 = a * c$ — contribution of the high parts.
- $p2 = b * d$ — contribution of the low parts.
- $p3 = (a+b) * (c+d)$ — the sum product.

Combine:

- $p4 = p3 - p1 - p2$ gives $a*d + b*c$.
- Result = $p1 * 10^{(2k)} + p4 * 10^k + p2$.

String arithmetic (C++):

- `addStrings` and `subtractStrings` implement big integer arithmetic for arbitrary precision.
- `multiplyByPower10` appends zeros.

11. Complexity Analysis

Operation	Complexity
Time	$O(n^{\log_2 3}) \approx O(n^{1.585})$
Space	$O(n)$
Grade-school	$O(n^2)$

Why $n^{\log_2 3}$?

The recurrence is $T(n) = 3T(n/2) + O(n)$. By the Master Theorem, this gives $O(n^{\log_2 3})$.

12. Common Patterns

Pattern 1: Big Integer Multiplication

- **How to identify:** Multiply numbers that don't fit in native types.
- **Approach:** Karatsuba or FFT-based multiplication.
- **Example:** Multiply Strings (LeetCode 43).

Pattern 2: Polynomial Multiplication

- **How to identify:** Multiply two polynomials.
- **Approach:** Karatsuba works for polynomials too (same formula).
- **Example:** Polynomial multiplication in competitive programming.

13. Common Mistakes

- **Wrong split point:** $k = n/2$ (floor), not $n/2$ (ceil). The high part gets $n-k$ digits, low part gets k digits.
- **Leading zeros in split parts:** When splitting, the low part may have leading zeros. Must handle them correctly.
- **Subtraction of strings:** $p3 - p1 - p2$ may be negative if the numbers are not properly split. Ensure $p3 \geq p1 + p2$ (it always is, by the formula's correctness).
- **Overflow in base case:** Using `stoi` for strings longer than 18 digits will overflow.
- **Not handling different lengths:** Pad both numbers to the same length before splitting.

14. Edge Cases

- One or both numbers are 0: result is 0.
- Numbers with different number of digits: pad with leading zeros.
- Single digit numbers: handled by base case.
- Very large numbers (1000+ digits): string-based implementation handles this.

- Negative numbers: the algorithm as given works for positive numbers only. For negatives, handle sign separately.

15. Variations

Toom-Cook Multiplication

- Generalization of Karatsuba: split into k parts.
- Toom-3 (3-way split) is $O(n^{\log_3 5}) \approx O(n^{1.465})$.
- More complex but faster for very large numbers.

FFT-Based Multiplication (Schönhage-Strassen)

- Uses Fast Fourier Transform.
- $O(n \log n \log \log n)$ — the fastest known for very large numbers.
- Used in practice for numbers with millions of digits.

Grade-School Multiplication

- $O(n^2)$ — fine for small numbers.
- The standard algorithm taught in school.

16. Related Algorithms

Algorithm	Complexity	When to Choose
Grade-School	$O(n^2)$	Small numbers ($n < 100$ digits)
Karatsuba	$O(n^{1.585})$	Medium numbers (100-1000 digits)
Toom-Cook	$O(n^{1.465})$	Large numbers (1000-10000 digits)
FFT (Schönhage-Strassen)	$O(n \log n \log \log n)$	Very large numbers (> 10000 digits)

17. Practice Problems

Easy

1. **Multiply Strings** — LeetCode 43 → Big integer multiplication using grade-school or Karatsuba.
2. **Add Strings** — LeetCode 415 → Big integer addition (prerequisite).

Medium

3. **Karatsuba Multiplication** — SPOJ or GFG → Implement Karatsuba.
4. **Big Integer Multiplication** — Codeforces → Implement using any fast multiplication algorithm.

Hard

5. **Polynomial Multiplication** — Codeforces 1628C → Use FFT or Karatsuba.
6. **Large Number Multiplication** — Many platforms → Full implementation of fast multiplication.

18. Interview Explanation

"Karatsuba multiplication is a divide-and-conquer algorithm for multiplying large integers. It's based on the observation that to multiply two n -digit numbers, we can split each into two halves, compute three products instead of four, and combine them. The standard formula $(a \cdot 10^k + b)(c \cdot 10^k + d)$ requires four multiplications: ac , ad , bc , bd . Karatsuba noticed that $ad + bc = (a+b)(c+d) - ac - bd$, so we only need three multiplications: ac , bd , and $(a+b)(c+d)$. This gives a recurrence of $T(n) = 3T(n/2) + O(n)$, which solves to $O(n^{\log_2 3}) \approx O(n^{1.585})$. For numbers with hundreds of digits, this is significantly faster than the $O(n^2)$ grade-school method."

19. Revision Notes

- Split into high and low halves.
- 3 multiplications: $p1 = ac$, $p2 = bd$, $p3 = (a+b)(c+d)$.
- Result = $p1 \cdot 10^{(2k)} + (p3 - p1 - p2) \cdot 10^k + p2$.
- Complexity: $O(n^{\log_2 3}) \approx O(n^{1.585})$.
- Base case: small numbers $\rightarrow$ direct multiplication.
- For strings, implement add, subtract, and multiply by 10^k .

20. Final Cheat Sheet

Aspect	Detail
When to use	Multiply large integers (100+ digits)
Main operations	3 recursive multiplications, addition, subtraction, shift
Complexity	Time: $O(n^{1.585})$, Space: $O(n)$
Key code idea	$p1 \cdot 10^{(2k)} + (p3 - p1 - p2) \cdot 10^k + p2$
Edge cases	Zero, different lengths, single digit
Common mistake	Wrong split point, overflow in base case

Fast Exponentiation

1. Overview

Fast Exponentiation (also called **Exponentiation by Squaring** or **Binary Exponentiation**) is a technique to compute a^b (a raised to power b) in $O(\log b)$ time instead of the naive $O(b)$. It works by repeatedly squaring the base and reducing the exponent based on its binary representation.

2. Intuition

Simple explanation:

Instead of multiplying a by itself b times, we square a repeatedly and combine the results based on the binary representation of b .

Analogy:

To compute 3^{10} , instead of $3 \times 3 \times 3 \times \dots \times 3$ (10 times), we can write 10 in binary (1010_2):

- $3^{10} = 3^8 \times 3^2$
- Compute $3^2 = 9$, $3^4 = 9^2 = 81$, $3^8 = 81^2 = 6561$
- Then $3^{10} = 6561 \times 9 = 59049$

Step-by-step reasoning:

1. Write the exponent b in binary.
2. For each bit of b (from LSB to MSB):
 - If the bit is 1, multiply the result by the current power of a .
 - Square the current power of a (move to the next bit).
3. The result is the product of $a^{(2^i)}$ for each set bit i in b .

Why it works:

$a^b = a^{(b_0 \cdot 2^0 + b_1 \cdot 2^1 + b_2 \cdot 2^2 + \dots)} = a^{(b_0 \cdot 2^0)} \times a^{(b_1 \cdot 2^1)} \times a^{(b_2 \cdot 2^2)} \times \dots$ where b_i is the i -th bit of b . Since $a^{(2^i)}$ can be computed by squaring a i times, we can compute all needed powers in $O(\log b)$ time.

3. When to Use It

- When computing **large exponents** (b up to 10^9 , 10^{18} , or more).
- When computing **modular exponentiation** ($a^b \bmod m$).
- When computing **linear recurrences** (Fibonacci numbers in $O(\log n)$).
- When computing **matrix exponentiation** (applying linear transformations repeatedly).
- When computing **powers in modular arithmetic** for cryptography.
- When computing **geometric series** or **repeated squaring** in any form.

Common trigger phrases: "calculate a^b ", "modular exponentiation", " $a^b \bmod m$ ", "power of a number", "binary exponentiation".

4. When Not to Use It

- When b is **small** ($b < 10$) — the naive loop is simpler and may be faster.
- When you need **exact real number exponentiation** with fractional exponents — use `pow()` from `cmath`.
- When a is **0 or 1** — simple edge cases, but fast exponentiation still works.
- When you need to compute **a^b for many different bases but same exponent** — precomputation of powers may be more efficient.
- When **floating-point precision** matters — use `pow()` from `cmath` (but it's $O(1)$).

5. Core Concepts

Binary Representation of Exponent

- Any integer b can be written as a sum of powers of 2.
- Example: $13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0$.
- $a^{13} = a^8 \times a^4 \times a^1$.

Modular Exponentiation

- Compute $(a^b) \% m$ without overflow.

- Use the property: $(x * y) \% m = ((x \% m) * (y \% m)) \% m$.
- Critical for preventing integer overflow in intermediate multiplications.

Recursive vs Iterative

- **Recursive:** $a^b = (a^{(b/2)})^2$ if b is even, $a^b = a \times a^{(b-1)}$ if b is odd.
- **Iterative:** Process bits of b from LSB to MSB.
- Both are $O(\log b)$. The iterative version is usually preferred.

Matrix Exponentiation

- Extension of fast exponentiation to matrices.
- Used to compute linear recurrences (Fibonacci, etc.) in $O(\log n)$ time.
- The matrix is raised to power n , and the result gives the n -th term.

6. Step-by-Step Algorithm

Iterative Fast Exponentiation

```
fastPow(a, b):
1. result = 1
2. while b > 0:
    if b & 1:           // if current bit is 1
        result = result * a
    a = a * a           // square the base
    b = b >> 1         // move to next bit
3. return result
```

Modular Exponentiation

```
modPow(a, b, m):
1. result = 1
2. a = a % m
3. while b > 0:
    if b & 1:
        result = (result * a) % m
    a = (a * a) % m
    b = b >> 1
4. return result
```

7. Dry Run

Compute 3^{13}

Binary of 13: 1101_2 (bits: 1, 0, 1, 1 from MSB to LSB)

```

Step 1: result = 1, a = 3, b = 13 (11012)
  b & 1 = 1 → result = 1 * 3 = 3
  a = 32 = 9
  b = 13 >> 1 = 6 (1102)

Step 2: b = 6 (1102), b & 1 = 0
  a = 92 = 81
  b = 6 >> 1 = 3 (112)

Step 3: b = 3 (112), b & 1 = 1 → result = 3 * 81 = 243
  a = 812 = 6561
  b = 3 >> 1 = 1 (12)

Step 4: b = 1 (12), b & 1 = 1 → result = 243 * 6561 = 1,594,323
  a = 65612 = 43,046,721
  b = 1 >> 1 = 0

Result: 1,594,323

Check: 313 = 1,594,323 ✓

```

Compute $3^{13} \bmod 7$

```

a = 3, b = 13, m = 7

Step 1: result = 1, a = 3 % 7 = 3, b = 13
  b & 1 = 1 → result = (1 * 3) % 7 = 3
  a = (3 * 3) % 7 = 9 % 7 = 2
  b = 6

Step 2: b = 6, b & 1 = 0
  a = (2 * 2) % 7 = 4
  b = 3

Step 3: b = 3, b & 1 = 1 → result = (3 * 4) % 7 = 12 % 7 = 5
  a = (4 * 4) % 7 = 16 % 7 = 2
  b = 1

Step 4: b = 1, b & 1 = 1 → result = (5 * 2) % 7 = 10 % 7 = 3
  a = (2 * 2) % 7 = 4
  b = 0

Result: 3

Check: 313 = 1,594,323, 1,594,323 % 7 = 3 ✓

```

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// Fast exponentiation: a^b
ll fastPow(ll a, ll b) {
    ll result = 1;
    while (b > 0) {
        if (b & 1) {
            result = result * a;
        }
        a = a * a;
        b >>= 1;
    }
    return result;
}

// Modular exponentiation: (a^b) % m
ll modPow(ll a, ll b, ll m) {
    ll result = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) {
            result = (result * a) % m;
        }
        a = (a * a) % m;
        b >>= 1;
    }
    return result;
}

// Safe modular multiplication for very large numbers (a * b % m without overflow)
ll modMul(ll a, ll b, ll m) {
    ll result = 0;
    a %= m;
    while (b > 0) {
        if (b & 1) {
            result = (result + a) % m;
        }
        a = (a + a) % m;
        b >>= 1;
    }
    return result;
}

// Safe modular exponentiation for very large numbers
ll modPowSafe(ll a, ll b, ll m) {
    ll result = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) {
```

```

        result = modMul(result, a, m);
    }
    a = modMul(a, a, m);
    b >>= 1;
}
return result;
}

// Example usage
int main() {
    cout << "3^13 = " << fastPow(3, 13) << "\n";           // 1594323
    cout << "3^13 % 7 = " << modPow(3, 13, 7) << "\n";       // 3
    cout << "2^100 % 1000000007 = " << modPow(2, 100, 1000000007) << "\n";
    return 0;
}

```

9. Python Implementation

```

def fast_pow(a, b):
    result = 1
    while b > 0:
        if b & 1:
            result *= a
        a *= a
        b >>= 1
    return result

def mod_pow(a, b, m):
    result = 1
    a %= m
    while b > 0:
        if b & 1:
            result = (result * a) % m
        a = (a * a) % m
        b >>= 1
    return result

# Example usage
print(f"3^13 = {fast_pow(3, 13)}")           # 1594323
print(f"3^13 % 7 = {mod_pow(3, 13, 7)}")     # 3
print(f"2^100 % 1000000007 = {mod_pow(2, 100, 1000000007)}")

```

10. Code Explanation

fastPow:

- `result = 1` — multiplicative identity.
- `while (b > 0)` — process each bit of the exponent.

- `if (b & 1)` — check if the current least significant bit is 1.
- If the bit is 1, multiply `result` by the current `a` (which is a^{2^i}).
- `a = a * a` — square the base to get the next power $a^{2^{i+1}}$.
- `b >>= 1` — shift exponent right to process the next bit.

modPow:

- Same logic, but with modular arithmetic at each step.
- `a %= m` — reduce base modulo `m` initially.
- `(result * a) % m` — multiply and reduce modulo `m` at each step.
- This prevents overflow because `result * a` could be up to $(m-1)^2$.

modMul (for very large m):

- When `m` is close to 10^{18} , `(result * a)` could overflow even 64-bit integers.
- `modMul` uses binary multiplication (similar to fast exponentiation) to multiply without overflow.
- Only needed when `m > 232` and you're using 64-bit integers.

11. Complexity Analysis

Operation	Complexity
Time	$O(\log b)$
Space	$O(1)$

Comparison:

- Naive: $O(b)$ — impractical for $b > 10^7$.
- Fast exponentiation: $O(\log b)$ — works for b up to 10^{18} or more.

12. Common Patterns

Pattern 1: Modular Exponentiation

- **How to identify:** "Compute $a^b \bmod m$ ", "modular power", "large exponent".
- **Approach:** Standard fast exponentiation with modulo at each step.
- **Example:** Pow(x, n) (LeetCode 50), Modular Exponentiation (GFG).

Pattern 2: Matrix Exponentiation

- **How to identify:** "Find n-th Fibonacci number", "linear recurrence", "n-th term of sequence".
- **Approach:** Represent the recurrence as a matrix, compute matrix^n using fast exponentiation.
- **Example:** Fibonacci Number (LeetCode 509) using matrix exponentiation.

Pattern 3: Binary Exponentiation for GCD

- **How to identify:** Not directly related, but fast exponentiation logic is used in extended Euclidean algorithm.
- **Approach:** Use binary exponentiation for modular inverses via Fermat's little theorem.
- **Example:** Compute modular inverse: $a^{(m-2)} \% m$.

Pattern 4: Repeated Squaring for Geometric Series

- **How to identify:** "Sum of geometric series", " $a + a^2 + a^3 + \dots + a^n$ ".
- **Approach:** Use divide-and-conquer on the sum (similar to exponentiation).
- **Example:** Sum of a geometric progression modulo m .

13. Common Mistakes

- **Not handling modulo correctly:** `(result * a) % m` is fine, but `result * a` may overflow in languages with fixed-size integers. Use `(result % m) * (a % m) % m`.
- **Forgetting `a %= m` at the start:** If $a \geq m$, the intermediate values may overflow.
- **Using `b > 0` instead of `b >= 0`:** Correct, but make sure `b` is non-negative.
- **Not handling `b = 0`:** $a^0 = 1$ for any non-zero a . 0^0 is undefined (usually return 1 by convention).
- **Overflow in `a = a * a`:** For large a (near 10^9), $a*a$ overflows 64-bit. Use `modMul` or modular arithmetic.
- **Negative exponent:** Fast exponentiation only works for non-negative integer exponents.

14. Edge Cases

- `b = 0`: return 1 (by convention, $0^0 = 1$ in most programming contexts).
- `a = 0`: return 0 (for $b > 0$).
- `a = 1`: return 1 (for any b).
- `b = 1`: return `a`.
- `m = 1`: result is 0 (anything mod 1 is 0).
- Large `b` (up to 10^{18}): works in $O(\log b) \approx 60$ iterations.
- Very large `b` ($b > 10^{18}$ in Python): Python handles arbitrary integers, but the loop runs $O(\log b)$ times.

15. Variations

Recursive Fast Exponentiation

```
fastPow(a, b):
    if b == 0: return 1
    half = fastPow(a, b / 2)
    if b % 2 == 0: return half * half
    else: return a * half * half
```

- Simpler to understand, but the iterative version is usually preferred.
- $O(\log b)$ time, $O(\log b)$ stack space.

Binary Exponentiation for Matrices

- Same algorithm, but operates on matrices.
- Matrix multiplication is $O(n^3)$ for $n \times n$ matrices.
- Used for linear recurrences (Fibonacci, etc.).

Exponentiation by Squaring with Precomputation

- Precompute $a^{(2^0)}, a^{(2^1)}, a^{(2^2)}, \dots, a^{(2^k)}$.
- Then combine for any exponent b by selecting the appropriate powers.
- Useful when computing a^b for many different b with the same a .

16. Related Algorithms

Algorithm	Difference	When to Choose
Naive Iteration	$O(b)$, simple	Small b ($b < 10^7$)
Fast Exponentiation	$O(\log b)$, iterative	General case, any b
Matrix Exponentiation	$O(n^3 \log b)$ for $n \times n$ matrices	Linear recurrences
pow() from cmath	$O(1)$, floating-point	Approximate real exponentiation
Exponentiation with Precomputation	$O(\log b)$ per query, $O(\log \text{MAX})$ precompute	Multiple queries with same base

17. Practice Problems

Easy

1. **Pow(x, n)** — LeetCode 50 → Implement fast exponentiation.
2. **Power of Two** — LeetCode 231 → Check if a number is a power of 2.

Medium

3. **Modular Exponentiation** — GFG → Compute $a^b \bmod m$.
4. **Super Pow** — LeetCode 372 → $a^b \bmod m$ where b is an array of digits.
5. **Fibonacci Number** — LeetCode 509 → Use matrix exponentiation for $O(\log n)$.

Hard

6. **K-th Ancestor of a Tree Node** — LeetCode 1483 → Binary lifting (uses fast exponentiation logic).
7. **Count Ways to Build Stairs** — Dynamic programming with matrix exponentiation.

18. Interview Explanation

"Fast exponentiation computes a^b in $O(\log b)$ time using the binary representation of the exponent. The key idea is that we can square the base repeatedly and combine results based on the set bits of the exponent. For example, $a^{13} = a^8 \times a^4 \times a^1$, and we compute a^2, a^4, a^8 by repeated squaring. The iterative version maintains a result variable, checks the least significant bit of b , multiplies result by the current power if the bit is set, squares the base, and shifts b right by 1. This loop runs $O(\log b)$ times. The same algorithm extends to modular exponentiation by adding modulo at each step, which is essential for preventing overflow in cryptographic applications."

19. Revision Notes

- `result = 1`, iterate while `b > 0`.

- If $b \& 1$, $result *= a$ (or $result = (result * a) \% m$).
- $a = a * a$ (or $a = (a * a) \% m$).
- $b \gg= 1$.
- $O(\log b)$ time, $O(1)$ space.
- Modular exponentiation: apply $\% m$ at every multiplication.
- Matrix exponentiation: same algorithm, matrix multiplication.
- Edge cases: $b = 0 \rightarrow 1$, $a = 0 \rightarrow 0$, $m = 1 \rightarrow 0$.

20. Final Cheat Sheet

Aspect	Detail
When to use	Compute a^b for large b , modular exponentiation, matrix exponentiation
Main operations	Bit check, multiply, square, shift
Complexity	Time: $O(\log b)$, Space: $O(1)$
Key code idea	<pre>while(b) { if(b&1) r*=a; a*=a; b>>=1; }</pre>
Edge cases	$b=0$, $a=0$, $m=1$, negative exponent
Common mistake	Overflow in $a*a$ without modulo, forgetting $a \%= m$

FFT (Fast Fourier Transform)

1. Overview

The Fast Fourier Transform (FFT) is an algorithm that computes the **Discrete Fourier Transform (DFT)** of a sequence in $O(n \log n)$ time instead of $O(n^2)$. It is one of the most important algorithms in signal processing, and in competitive programming, it is primarily used for **polynomial multiplication** (convolution) and for solving problems involving large-integer multiplication, string matching, and more.

2. Intuition

Simple explanation:

FFT converts a sequence of values (like polynomial coefficients) from the time/coefficient domain to the frequency/point-value domain, and vice versa, in $O(n \log n)$ time. This allows us to multiply polynomials efficiently by evaluating them at special points (roots of unity), multiplying pointwise, and converting back.

Analogy:

Multiplying two polynomials the normal way (FOIL) is $O(n^2)$. But if you evaluate both polynomials at enough points, multiply the values at each point, and then interpolate to get back the coefficients — you can do this in $O(n \log n)$ using FFT. The trick is choosing the "right" points (roots of unity) that make the evaluation and interpolation fast.

Step-by-step reasoning:

1. Represent polynomials as arrays of coefficients.
2. Pad them to the same size (power of 2).

3. Compute the FFT of both polynomials (convert to point-value form).
4. Multiply the point values pointwise.
5. Compute the inverse FFT (convert back to coefficient form).
6. Round the results (since floating-point arithmetic introduces small errors).

Why it works (simplified):

The DFT uses the n -th roots of unity as evaluation points. These points have the property that evaluating a polynomial at all n roots can be done in $O(n \log n)$ using the divide-and-conquer FFT algorithm, which exploits the symmetry of the roots.

3. When to Use It

- When multiplying **two polynomials** of degree n (convolution).
- When multiplying **two large integers** (using FFT-based multiplication).
- When solving problems involving **convolution** of two sequences.
- When doing **string matching with wildcards**.
- When computing **all pairwise sums/differences** of two arrays.
- When computing **correlation** or **autocorrelation**.
- When solving **subset sum** problems with large constraints.
- When you need to multiply **multiple polynomials** together.

Common trigger phrases: "polynomial multiplication", "convolution", "multiply large numbers", "convolution modulo", "FFT", "NTT", "number theoretic transform".

4. When Not to Use It

- When the polynomials are **small** (degree < 1000) — $O(n^2)$ is simpler and may be faster due to constants.
- When you need **exact integer results** — floating-point FFT can have precision issues. Use NTT (Number Theoretic Transform) with a modulus.
- When the **modulus is not compatible** with NTT (e.g., not a prime of form $k \cdot 2^n + 1$).
- When you need to multiply **more than 2 polynomials** — consider using divide-and-conquer with FFT.
- When the problem can be solved with simpler techniques (e.g., frequency arrays for small ranges).

5. Core Concepts

Complex Roots of Unity

- The n -th roots of unity are the solutions to $\omega^n = 1$.
- They are $\omega_k = e^{(2\pi i k/n)} = \cos(2\pi k/n) + i \sin(2\pi k/n)$.
- They form a cyclic group under multiplication.
- **Key property:** $\omega_{-(n/2)}^{(n/2)} = -1$, which enables the divide-and-conquer.

DFT (Discrete Fourier Transform)

- Converts a polynomial from coefficient form to point-value form.
- Evaluates the polynomial at all n -th roots of unity.
- Naive: $O(n^2)$, FFT: $O(n \log n)$.

Inverse DFT

- Converts back from point-value form to coefficient form.
- Uses the same FFT algorithm with conjugated roots and a normalization factor.

Convolution

- $(a * b)[k] = \sum_i a[i] \cdot b[k-i]$ for all valid i .
- This is exactly polynomial multiplication.
- Using FFT: compute FFT of both arrays, multiply pointwise, compute inverse FFT.

NTT (Number Theoretic Transform)

- FFT in a finite field modulo a prime.
- Uses primitive roots of unity modulo p instead of complex numbers.
- Avoids floating-point precision issues.
- Requires a prime of the form $p = c \cdot 2^n + 1$ (e.g., $998244353 = 119 \cdot 2^{23} + 1$).

6. Step-by-Step Algorithm

FFT (Cooley-Tukey, Iterative)

```
fft(a, invert):
  1. n = a.size()
  2. Rearrange a using bit-reversal permutation
  3. for len = 2 to n, len *= 2:
      ang = 2 * PI / len * (invert ? -1 : 1)
      wlen = complex(cos(ang), sin(ang))
      for i = 0 to n, i += len:
          w = 1
          for j = 0 to len/2:
              u = a[i + j]
              v = a[i + j + len/2] * w
              a[i + j] = u + v
              a[i + j + len/2] = u - v
              w *= wlen
  4. if invert:
      for each a[i]: a[i] /= n
```

Polynomial Multiplication

```
multiply(a, b):
  1. n = 1
     while n < a.size() + b.size() - 1: n *= 2
  2. Pad a and b to size n
  3. fft(a, false)
  4. fft(b, false)
  5. for i = 0 to n: a[i] *= b[i]
  6. fft(a, true)
```

7. Round real parts to integers
8. Return a (trimmed to correct size)

7. Dry Run

Multiply polynomials: $(1 + 2x) \times (3 + 4x)$

Coefficients: $a = [1, 2]$, $b = [3, 4]$

Expected: $(1+2x)(3+4x) = 3 + 10x + 8x^2 \rightarrow [3, 10, 8]$

Step 1: $n = 1$, while $n < 2+2-1=3$: $n = 4$

Step 2: Pad: $a = [1, 2, 0, 0]$, $b = [3, 4, 0, 0]$

Step 3: FFT of a (not fully traced, but conceptually):

$[1, 2, 0, 0] \rightarrow \text{FFT} \rightarrow [1+0i, 1-2i, 1+0i, 1+2i]$ (approximately)

(Actually let me not compute the exact complex values here)

Step 4: FFT of b : $[3, 4, 0, 0] \rightarrow \text{FFT} \rightarrow [3+0i, 3-4i, 3+0i, 3+4i]$

Step 5: Pointwise multiply:

$$a[0]*b[0] = (1+0i)(3+0i) = 3$$

$$a[1]*b[1] = (1-2i)(3-4i) = 3 - 4i - 6i + 8i^2 = 3 - 10i - 8 = -5 - 10i$$

$$a[2]*b[2] = (1+0i)(3+0i) = 3$$

$$a[3]*b[3] = (1+2i)(3+4i) = 3 + 4i + 6i + 8i^2 = 3 + 10i - 8 = -5 + 10i$$

Step 6: Inverse FFT of result:

$$[3, -5-10i, 3, -5+10i] \rightarrow \text{inverse FFT} \rightarrow [3, 10, 8, 0]$$

Step 7: Round: $[3, 10, 8, 0]$

Step 8: Trim to size 3: $[3, 10, 8]$ ✓

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd>& a, bool invert) {
    int n = a.size();

    // Bit-reversal permutation
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
}
```

```

// Cooley-Tukey iterative FFT
for (int len = 2; len <= n; len <= 1) {
    double ang = 2 * PI / len * (invert ? -1 : 1);
    cd wlen(cos(ang), sin(ang));

    for (int i = 0; i < n; i += len) {
        cd w(1);
        for (int j = 0; j < len / 2; j++) {
            cd u = a[i + j];
            cd v = a[i + j + len / 2] * w;
            a[i + j] = u + v;
            a[i + j + len / 2] = u - v;
            w *= wlen;
        }
    }
}

if (invert) {
    for (cd& x : a) x /= n;
}
}

// Multiply two polynomials given as coefficient vectors
vector<int> multiplyPolynomials(const vector<int>& a, const vector<int>& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());

    int n = 1;
    while (n < (int)(a.size() + b.size() - 1)) n <= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);

    for (int i = 0; i < n; i++) fa[i] *= fb[i];

    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++) {
        result[i] = round(fa[i].real());
    }

    // Remove trailing zeros
    while (result.size() > 1 && result.back() == 0) result.pop_back();
    return result;
}

// Example usage
int main() {
    vector<int> a = {1, 2}; // 1 + 2x
    vector<int> b = {3, 4}; // 3 + 4x

```

```

vector<int> result = multiplyPolynomials(a, b);

cout << "Result: ";
for (int x : result) cout << x << " ";
cout << "\n"; // 3 10 8
return 0;
}

```

9. Python Implementation

```

import cmath
import math

def fft(a, invert):
    n = len(a)

    # Bit-reversal permutation
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            a[i], a[j] = a[j], a[i]

    # Cooley-Tukey iterative FFT
    length = 2
    while length <= n:
        ang = 2 * math.pi / length * (-1 if invert else 1)
        wlen = complex(math.cos(ang), math.sin(ang))

        for i in range(0, n, length):
            w = 1 + 0j
            half = length // 2
            for j in range(half):
                u = a[i + j]
                v = a[i + j + half] * w
                a[i + j] = u + v
                a[i + j + half] = u - v
                w *= wlen

            length <<= 1

    if invert:
        for i in range(n):
            a[i] /= n

```

```
def multiply_polynomials(a, b):
    fa = [complex(x, 0) for x in a]
    fb = [complex(x, 0) for x in b]

    n = 1
    while n < len(a) + len(b) - 1:
        n <= 1

    fa.extend([0j] * (n - len(fa)))
    fb.extend([0j] * (n - len(fb)))

    fft(fa, False)
    fft(fb, False)

    for i in range(n):
        fa[i] *= fb[i]

    fft(fa, True)

    result = [round(fa[i].real) for i in range(n)]

    # Remove trailing zeros
    while len(result) > 1 and result[-1] == 0:
        result.pop()

    return result

# Example usage
a = [1, 2] # 1 + 2x
b = [3, 4] # 3 + 4x
result = multiply_polynomials(a, b)
print(f"Result: {result}") # [3, 10, 8]
```

10. Code Explanation

Bit-reversal permutation:

- FFT processes pairs of elements that are $\text{len}/2$ apart.
- The iterative FFT requires elements to be in bit-reversed order.
- Example: $n=8$, index 3 (binary 011) becomes 6 (binary 110).

Main FFT loop:

- len is the current subproblem size (2, 4, 8, ..., n).
- w_{len} is the primitive len -th root of unity.
- For each block of size len , we combine pairs $(i+j, i+j+\text{len}/2)$.
- $u + v$ and $u - v$ is the butterfly operation.

Inverse FFT:

- Same algorithm, but with conjugated roots ($\text{ang} = -2*\text{PI}/\text{len}$).

- After the loop, divide each element by n .

Polynomial multiplication:

- Convert coefficients to complex numbers.
- Pad to power of 2.
- FFT both, multiply pointwise, inverse FFT.
- Round and trim.

11. Complexity Analysis

Operation	Complexity
FFT (forward or inverse)	$O(n \log n)$
Polynomial multiplication	$O(n \log n)$
Naive polynomial multiplication	$O(n^2)$
Space	$O(n)$

12. Common Patterns

Pattern 1: Polynomial Multiplication (Convolution)

- **How to identify:** "Multiply polynomials", "convolution of two arrays".
- **Approach:** FFT both, multiply, inverse FFT.
- **Example:** Multiply two polynomials (GFG), Convolution (many CP platforms).

Pattern 2: Large Integer Multiplication

- **How to identify:** "Multiply two large numbers", "big integer multiplication".
- **Approach:** Treat digits as coefficients, multiply polynomials, handle carries.
- **Example:** Multiply Strings (LeetCode 43) — can be solved with FFT for very large numbers.

Pattern 3: String Matching with Wildcards

- **How to identify:** "Pattern matching with wildcards", "fuzzy matching".
- **Approach:** Map characters to numbers, use convolution to find matches.
- **Example:** Wildcard Matching using FFT.

Pattern 4: All Pairwise Sums/Differences

- **How to identify:** "Count pairs with sum/difference in range", "subset sum with large values".
- **Approach:** Use convolution to compute the frequency of all sums.
- **Example:** Count pairs with given sum (large n).

Pattern 5: Multiple Polynomial Multiplication

- **How to identify:** "Multiply K polynomials", "product of many polynomials".
- **Approach:** Use divide-and-conquer with FFT (multiply pairs, then combine).
- **Example:** Product of polynomials (Codeforces).

13. Common Mistakes

- **Not padding to power of 2:** FFT requires n to be a power of 2.
- **Off-by-one in result size:** Result size is `a.size() + b.size() - 1`.
- **Floating-point precision:** Rounding errors can cause wrong results. Use `round()` before converting to `int`.
- **Not handling negative coefficients:** Works fine with complex numbers.
- **Forgetting to reset arrays:** FFT modifies the input arrays.
- **Wrong primitive root for NTT:** For NTT, the primitive root must be chosen correctly based on the modulus.
- **Stack overflow in recursive FFT:** Always use the iterative implementation.

14. Edge Cases

- Empty polynomials: return empty result.
- Single coefficient: direct multiplication.
- Zero polynomial: result is zero.
- Large coefficients: floating-point precision may fail for very large values.
- Negative coefficients: works fine.
- Non-power-of-2 size: pad to the next power of 2.

15. Variations

NTT (Number Theoretic Transform)

- FFT in a finite field modulo a prime.
- Uses modular arithmetic, no floating-point errors.
- Requires a prime of form $p = c \cdot 2^n + 1$.
- Common modulus: 998244353 ($119 \cdot 2^{23} + 1$), 1004535809 ($479 \cdot 2^{21} + 1$).

Recursive FFT

- Easier to understand but may cause stack overflow.
- Same complexity as iterative FFT.

Three-Step FFT (for large n)

- Splits the problem into smaller FFTs.
- Used when n is very large and memory is limited.

Bluestein's Algorithm

- Computes FFT of any size (not just power of 2).
- Uses convolution with a chirp signal.

16. Related Algorithms

Algorithm	Difference	When to Choose
Naive $O(n^2)$	Simple, no overhead	Small polynomials ($n < 1000$)

Algorithm	Difference	When to Choose
FFT (Complex)	$O(n \log n)$, floating-point	General case, moderate precision
NTT	$O(n \log n)$, exact integer	When exact integer results are needed
Karatsuba	$O(n^{1.585})$, exact integer	For integer multiplication, simpler than FFT
Convolution using DP	$O(n^2)$	Small constraints, specific patterns

17. Practice Problems

Easy

1. **Multiply Polynomials** — GFG → Basic polynomial multiplication.
2. **Convolution** — CSES → Compute convolution of two arrays.

Medium

3. **Polynomial Multiplication (FFT)** — Codeforces 632E → FFT-based polynomial multiplication.
4. **Match the Strings** — Custom → String matching with wildcards using FFT.

Hard

5. **Large Number Multiplication** — Many platforms → Multiply large integers using FFT.
6. **Product of Polynomials** — Codeforces 286E → Multiple polynomial multiplication.
7. **FFT with Modulo (NTT)** — Many platforms → Implement NTT for exact results.

18. Interview Explanation

"The Fast Fourier Transform converts a polynomial from coefficient form to point-value form in $O(n \log n)$ time. This is useful because multiplying two polynomials is $O(n^2)$ in coefficient form but only $O(n)$ in point-value form. The FFT evaluates the polynomial at the n -th roots of unity, which have special symmetry properties that allow the divide-and-conquer approach. The iterative Cooley-Tukey algorithm works by repeatedly combining pairs of values using the butterfly operation. To multiply polynomials: convert both to point-value form using FFT, multiply pointwise, and convert back using inverse FFT. For exact integer results, I'd use the Number Theoretic Transform (NTT) with a suitable prime modulus to avoid floating-point precision issues."

19. Revision Notes

- FFT: $O(n \log n)$ for DFT/IDFT.
- Bit-reversal permutation before the main loop.
- Butterfly operation: $u + v, u - v$.
- Inverse FFT: use conjugated roots, divide by n at the end.
- Polynomial multiplication: FFT → multiply → inverse FFT.
- NTT: same algorithm, but in a finite field (mod prime).
- Common modulus: 998244353 (primitive root 3).
- Only use when $n > 1000$ (otherwise $O(n^2)$ is fine).

20. Final Cheat Sheet

Aspect	Detail
When to use	Polynomial multiplication, convolution, large integer multiplication, string matching
Main operations	Bit-reversal, butterfly (u+v, u-v), pointwise multiply
Complexity	Time: $O(n \log n)$, Space: $O(n)$
Key code idea	<code>fft(a, invert), multiply(a, b) → fft → pointwise → ifft</code>
Edge cases	Empty, single, non-power-of-2, large coefficients
Common mistake	Precision issues — use NTT for exact integer results

CDQ Divide and Conquer

1. Overview

CDQ Divide and Conquer (named after Chen Danqi, the Chinese competitive programmer who popularized it) is a technique for solving **offline dynamic programming problems** and **multidimensional partial order problems**. It is a divide-and-conquer approach where the left half's results are used to update the right half. It is particularly powerful for solving **3D/4D partial order** problems (like counting elements that are smaller in all dimensions) and **offline dynamic programming** problems where DP transitions depend on previous states.

2. Intuition

Simple explanation:

CDQ divide and conquer is a technique where you solve a problem by processing the array in order, dividing it into two halves, recursively solving the left half, then using the left half's results to update the right half (using the "divide and conquer on time" principle), and finally recursively solving the right half.

Analogy:

Imagine you're tracking events in time. Each event has a "time" and some other attributes. You want to compute something for each event based on all previous events. Instead of checking all previous events ($O(n^2)$), you split the timeline, solve the left half, then figure out how the left half contributes to the right half, and then solve the right half.

Step-by-step reasoning:

1. Divide the array (or event list) into two halves by index (or by time).
2. Recursively solve the left half.
3. Process the contribution of the left half to the right half:
 - Sort both halves by a secondary dimension.
 - Use a Fenwick tree (BIT) or other data structure to process the "left" elements and update the "right" elements.
4. Recursively solve the right half.
5. (Optional) Merge the two halves back to the original order.

Why it works:

CDQ divide and conquer reduces a k -dimensional problem to a $(k-1)$ -dimensional problem plus a divide-and-conquer step. Each level of recursion adds one dimension, and the total complexity is $O(n \log^k n)$ for k -dimensional problems.

3. When to Use It

- When solving **3D partial order** problems (count/max/sum over elements smaller in all dimensions).
- When solving **offline dynamic programming** where $DP[i]$ depends on $DP[j]$ for $j < i$ and some condition.
- When solving **2D range queries** offline (like counting points in a rectangle).
- When solving **CDQ + DP** problems (like finding the longest increasing subsequence in 3D).
- When the problem can be modeled as "**contribution of left to right**".
- When you need to **reduce the dimension** of a problem ($k-D \rightarrow (k-1)-D$).

Common trigger phrases: "3D partial order", "count points in 3D space", "offline DP", "add points and query rectangles", "CDQ divide and conquer", "multidimensional BIT".

4. When Not to Use It

- When the problem is **online** (queries arrive in real time) — CDQ only works for offline problems.
- When the problem is **1D or 2D simple** — sorting or BIT alone is sufficient.
- When the data is **small** ($n \leq 1000$) — $O(n^2)$ or $O(n^2 \log n)$ may be simpler.
- When the problem involves **updates to the data** — CDQ works on static data (or offline sequence of events).
- When a **segment tree with lazy propagation** or **Fenwick tree** is directly applicable.

5. Core Concepts

K-Dimensional Partial Order

- A tuple $(x_1, x_2, \dots, x_k)$ is said to dominate another tuple if all dimensions are $\leq$ (or $\geq$).
- Problem: for each element, count how many elements dominate it.
- 1D: sort ($O(n \log n)$).
- 2D: sort by x , use BIT for y ($O(n \log n)$).
- 3D: CDQ divide and conquer + BIT ($O(n \log^2 n)$).

Offline Processing

- All queries and data are known in advance.
- The algorithm processes them in a specific order (not necessarily the input order).
- CDQ reorders events by time, processes contributions, then restores order.

Time Dimension

- In CDQ, the "first dimension" is usually the index (time) in the array.
- The divide is always by index.
- The left half always happens "before" the right half.

Contribution from Left to Right

- After solving the left half recursively, we compute how the left half affects the right half.
- This is done by sorting both halves by a secondary dimension and using a BIT.
- This is the "combine" step in CDQ.

CDQ + DP

- $DP[i] = \max/\min/\text{sum over } DP[j] + \text{cost}(j, i)$ where $j < i$ and some condition holds.
- CDQ divides by index, solves left DP, then uses left to update right DP, then solves right DP.
- This is equivalent to using CDQ for the "time" dimension.

6. Step-by-Step Algorithm

3D Partial Order Counting

Problem: Count, for each element (x, y, z) , how many elements have $x_i < x, y_i < y, z_i < z$.

```
cdq(elements, left, right):
  1. If left >= right: return
  2. mid = (left + right) / 2
  3. cdq(elements, left, mid)      // solve left half
  4. // Process left → right contributions
     - Create a temporary array with elements from [left, right]
     - Sort temp by y
     - For each element in temp (sorted by y):
         if element is from left half:
             BIT.add(element.z, count)
         else:
             result[element] += BIT.query(element.z - 1)
     - Clear BIT (reset added values)
  5. cdq(elements, mid + 1, right) // solve right half
  6. Merge elements[left..right] by x (or restore original order)
```

CDQ + DP (LIS in 3D)

Problem: $DP[i] = \max \text{ over } DP[j] + 1$ where $j < i, x_j < x_i, y_j < y_i$

```
cdq(dp, elements, left, right):
  1. If left >= right: return
  2. mid = (left + right) / 2
  3. cdq(dp, elements, left, mid)      // solve left DP
  4. // Use left DP to update right DP
     - Sort elements[left..right] by x
     - For each element in sorted order:
         if element is from left half:
             BIT.add(element.y, dp[element])
         else:
             dp[element] = BIT.query(element.y) + 1
```

```

        dp[element] = max(dp[element], BIT.query(element.y - 1) + 1)
    - Clear BIT
5. cdq(dp, elements, mid + 1, right) // solve right DP
6. Merge elements[left..right] by original index

```

7. Dry Run

3D Partial Order Count

Input: $[(1, 2, 3), (2, 1, 2), (1, 3, 1), (2, 2, 2)]$ Goal: For each element, count how many have smaller x, y, z .

Sorted by x : $[(1,2,3), (1,3,1), (2,1,2), (2,2,2)]$
 $[idx:0, \quad idx:2, \quad idx:1, \quad idx:3]$

Step 1: $cdq(elements, 0, 3)$
 $mid = 1$

Step 2: $cdq(elements, 0, 1)$ – left half
 $elements[0..1] = [(1,2,3), (1,3,1)]$
 $mid = 0$
 $cdq(0,0)$ – single element, return
 $cdq(1,1)$ – single element, return

Process left→right for half $[0,1]$:
Sort by y : $(1,2,3)$ [left], $(1,3,1)$ [right]
Process $(1,2,3)$: left → $BIT.add(z=3, 1)$
Process $(1,3,1)$: right → $BIT.query(z-1=0) = 0$
Clear BIT
 $result[idx=2] = 0$ (no element with smaller y and z)

Step 3: $cdq(elements, 2, 3)$ – right half
 $elements[2..3] = [(2,1,2), (2,2,2)]$
 $mid = 2$
 $cdq(2,2)$ – single element, return
 $cdq(3,3)$ – single element, return

Process left→right for half $[2,3]$:
Sort by y : $(2,1,2)$ [left], $(2,2,2)$ [right]
Process $(2,1,2)$: left → $BIT.add(z=2, 1)$
Process $(2,2,2)$: right → $BIT.query(z-1=1) = 0$
Clear BIT
 $result[idx=3] = 0$

Step 4: Process left→right for full range $[0,3]$:
Elements sorted by y : $(2,1,2)$ [right], $(1,2,3)$ [left], $(2,2,2)$ [right], $(1,3,1)$ [left]
Process $(2,1,2)$: right → $BIT.query(1) = 0$
Process $(1,2,3)$: left → $BIT.add(3, 1)$
Process $(2,2,2)$: right → $BIT.query(1) = 0$
Process $(1,3,1)$: left → $BIT.add(1, 1)$

Clear BIT

Result: [0, 0, 0, 0]

Wait, let me reconsider. The elements order by x was: idx=0: (1,2,3) idx=2: (1,3,1) idx=1: (2,1,2) idx=3: (2,2,2)

For the cross contribution of the full range: Left half: idx=0 (1,2,3), idx=2 (1,3,1) Right half: idx=1 (2,1,2), idx=3 (2,2,2)

Sort by y: (2,1,2) — right — BIT.query(1) = 0 (1,2,3) — left — BIT.add(3, 1) (2,2,2) — right — BIT.query(1) = 0
(we need $z < 2$, so $z=1$, BIT.query(1)=0) (1,3,1) — left — BIT.add(1, 1)

So all results are 0. Let me check if any element is smaller in all dimensions:

- (1,2,3) vs (2,1,2): $x=1 < 2$ ✓, $y=2 > 1$ ✗, $z=3 > 2$ ✗ → no
- (1,2,3) vs (2,2,2): $x=1 < 2$ ✓, $y=2=2$ (not <), $z=3 > 2$ ✗ → no
- (1,3,1) vs (2,1,2): $x=1 < 2$ ✓, $y=3 > 1$ ✗ → no
- (1,3,1) vs (2,2,2): $x=1 < 2$ ✓, $y=3 > 2$ ✗ → no
- (1,2,3) vs (1,3,1): $x=1=1$ ✗ → no
- (2,1,2) vs (2,2,2): $x=2=2$ ✗ → no

So indeed all results are 0. That's correct for this input.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Fenwick Tree (Binary Indexed Tree)
struct BIT {
    int n;
    vector<int> tree;

    BIT(int n) : n(n), tree(n + 1, 0) {}

    void add(int idx, int val) {
        for (; idx <= n; idx += idx & -idx)
            tree[idx] += val;
    }

    int sum(int idx) {
        int res = 0;
        for (; idx > 0; idx -= idx & -idx)
            res += tree[idx];
        return res;
    }

    void clear() {
        fill(tree.begin(), tree.end(), 0);
    }
};
```

```

struct Element {
    int x, y, z;
    int id;      // original index
    int ans;     // result for this element
};

// 3D partial order count using CDQ divide and conquer
void cdq3D(vector<Element>& elements, int left, int right, BIT& bit) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    // Solve left half
    cdq3D(elements, left, mid, bit);

    // Process left → right contributions
    // Create a temporary vector for elements in [left, right]
    vector<Element> temp;
    for (int i = left; i <= right; i++) {
        temp.push_back(elements[i]);
    }

    // Sort by y
    sort(temp.begin(), temp.end(), [](const Element& a, const Element& b) {
        if (a.y != b.y) return a.y < b.y;
        return a.z < b.z; // tie-break by z
    });

    for (const auto& e : temp) {
        if (e.id <= mid) {
            // Element from left half: add to BIT
            bit.add(e.z, 1);
        } else {
            // Element from right half: query BIT
            elements[e.id].ans += bit.sum(e.z - 1);
        }
    }

    // Clear BIT (reset the values we added)
    for (const auto& e : temp) {
        if (e.id <= mid) {
            bit.add(e.z, -1);
        }
    }

    // Solve right half
    cdq3D(elements, mid + 1, right, bit);
}

// Example usage: Count 3D partial order
int main() {
    int n = 4;
    vector<Element> elements(n);

```

```

// Initialize elements
elements[0] = {1, 2, 3, 0, 0};
elements[1] = {2, 1, 2, 1, 0};
elements[2] = {1, 3, 1, 2, 0};
elements[3] = {2, 2, 2, 3, 0};

// Sort by x first
sort(elements.begin(), elements.end(), [](const Element& a, const Element& b)
{
    if (a.x != b.x) return a.x < b.x;
    if (a.y != b.y) return a.y < b.y;
    return a.z < b.z;
}));

// Update IDs after sorting by x
for (int i = 0; i < n; i++) {
    elements[i].id = i;
}

int maxZ = 3; // maximum z value (for BIT size)
BIT bit(maxZ);

cdq3D(elements, 0, n - 1, bit);

// Output results
for (int i = 0; i < n; i++) {
    cout << "Element (" << elements[i].x << ", "
        << elements[i].y << ", " << elements[i].z
        << "): " << elements[i].ans << "\n";
}

return 0;
}

```

CDQ + DP (3D LIS)

```

#include <bits/stdc++.h>
using namespace std;

struct BIT {
    int n;
    vector<int> tree;

    BIT(int n) : n(n), tree(n + 1, 0) {}

    void update(int idx, int val) {
        for (; idx <= n; idx += idx & -idx)
            tree[idx] = max(tree[idx], val);
    }

    int query(int idx) {

```

```

        int res = 0;
        for (; idx > 0; idx -= idx & -idx)
            res = max(res, tree[idx]);
        return res;
    }

    void reset(int idx) {
        for (; idx <= n; idx += idx & -idx)
            tree[idx] = 0;
    }
};

struct Element {
    int x, y, z, id, dp;
};

void cdqDP(vector<Element>& elements, int left, int right, BIT& bit) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    // Solve left half
    cdqDP(elements, left, mid, bit);

    // Sort by x for the current range
    vector<Element> temp(elements.begin() + left, elements.begin() + right + 1);
    sort(temp.begin(), temp.end(), [](const Element& a, const Element& b) {
        return a.x < b.x;
    });

    // Process left → right contributions
    for (auto& e : temp) {
        if (e.id <= mid) {
            bit.update(e.y, e.dp);
        } else {
            elements[e.id].dp = max(elements[e.id].dp, bit.query(e.y - 1) + 1);
        }
    }

    // Reset BIT
    for (auto& e : temp) {
        if (e.id <= mid) {
            bit.reset(e.y);
        }
    }

    // Solve right half
    cdqDP(elements, mid + 1, right, bit);
}

int main() {
    int n = 4;
    vector<Element> elements(n);
    elements[0] = {1, 2, 3, 0, 1};
}

```

```

elements[1] = {2, 1, 2, 1, 1};
elements[2] = {1, 3, 1, 2, 1};
elements[3] = {2, 2, 2, 3, 1};

// Sort by x
sort(elements.begin(), elements.end(), [](const Element& a, const Element& b)
{
    return a.x < b.x;
});
for (int i = 0; i < n; i++) elements[i].id = i;

BIT bit(3); // max y = 3
cdqDP(elements, 0, n - 1, bit);

int ans = 0;
for (auto& e : elements) ans = max(ans, e.dp);
cout << "Longest 3D LIS length: " << ans << "\n";

return 0;
}

```

9. Python Implementation

```

class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (n + 1)

    def add(self, idx, val):
        while idx <= self.n:
            self.tree[idx] += val
            idx += idx & -idx

    def sum(self, idx):
        res = 0
        while idx > 0:
            res += self.tree[idx]
            idx -= idx & -idx
        return res

    def clear(self):
        self.tree = [0] * (self.n + 1)

def cdq_3d(elements, left, right, bit):
    if left >= right:
        return

    mid = (left + right) // 2

    cdq_3d(elements, left, mid, bit)

```

```

# Process left → right contributions
temp = elements[left:right + 1]
temp.sort(key=lambda e: (e[1], e[2])) # sort by y, then z

for e in temp:
    idx, y, z = e[3], e[1], e[2]
    if idx <= mid:
        bit.add(z, 1)
    else:
        elements[idx][4] += bit.sum(z - 1)

# Clear BIT
for e in temp:
    if e[3] <= mid:
        bit.add(e[2], -1)

cdq_3d(elements, mid + 1, right, bit)

# Example usage: [x, y, z, id, ans]
elements = [
    [1, 2, 3, 0, 0],
    [2, 1, 2, 1, 0],
    [1, 3, 1, 2, 0],
    [2, 2, 2, 3, 0],
]

# Sort by x
elements.sort(key=lambda e: (e[0], e[1], e[2]))
for i, e in enumerate(elements):
    e[3] = i

bit = BIT(3) # max z
cdq_3d(elements, 0, len(elements) - 1, bit)

for e in elements:
    print(f"({e[0]}, {e[1]}, {e[2]}): {e[4]}")

```

10. Code Explanation

CDQ for 3D Partial Order:

- Elements are sorted by **x** initially.
- The **id** field stores the position in the sorted order.
- The divide step splits by **id** (which corresponds to the **x** dimension).
- The combine step sorts by **y** and uses a BIT on **z**.
- Left elements add their **z** to the BIT.
- Right elements query the BIT for **z** values smaller than their own.

CDQ + DP:

- Similar structure, but the BIT stores the maximum DP value instead of a count.
- The DP update happens in the combine step.
- Left elements update the BIT with their DP value at position y .
- Right elements query the BIT for the maximum DP among elements with smaller y .

Key insight: The BIT must be cleared after each combine step. In the counting version, we subtract the added values. In the DP version, we reset the specific positions to 0.

11. Complexity Analysis

Operation	Complexity
3D Partial Order (CDQ + BIT)	$O(n \log^2 n)$
4D Partial Order (CDQ + CDQ + BIT)	$O(n \log^3 n)$
CDQ + DP	$O(n \log^2 n)$
Space	$O(n)$

Recurrence: $T(n) = 2T(n/2) + O(n \log n) \rightarrow O(n \log^2 n)$.

12. Common Patterns

Pattern 1: 3D Partial Order Counting

- **How to identify:** "Count elements smaller in all three dimensions".
- **Approach:** Sort by x , CDQ by y , BIT by z .
- **Example:** Count of points in 3D space (many CP problems).

Pattern 2: CDQ + DP (3D LIS)

- **How to identify:** "Longest increasing subsequence in 3D", "DP with 3D constraints".
- **Approach:** CDQ for index dimension, BIT for the third dimension.
- **Example:** LIS in 3D space (POJ 1631, BOJ 2568).

Pattern 3: Offline Dynamic Programming

- **How to identify:** $DP[i] = \max \text{ over } DP[j] + \text{cost}(i, j)$ where $j < i$ and condition holds.
- **Approach:** CDQ divides by index, left updates right.
- **Example:** DP with " $j < i$ and $x_j < x_i$ " constraints.

Pattern 4: 2D Range Query using CDQ

- **How to identify:** "Add points, query rectangles", "offline 2D range queries".
- **Approach:** Convert each query into four prefix queries, CDQ by time.
- **Example:** Count points in a rectangle (offline).

13. Common Mistakes

- **Not resetting the BIT properly:** After the combine step, the BIT must be cleared. Use subtraction or explicit reset.

- **Incorrect sorting in combine step:** The combine step must sort by the secondary dimension (y), not by the primary dimension (x).
- **Not handling equal elements:** For partial order with strict inequality, handle equality carefully. Usually, sort by z in the combine step's tie-break.
- **Wrong order of recursion:** Must solve left, then combine, then solve right. Not left, right, combine.
- **Not updating IDs after sorting:** After sorting by x, the IDs must reflect the new positions.
- **Stack overflow:** Recursion depth is $O(\log n)$, so fine for n up to 10^6 .

14. Edge Cases

- Empty array: return immediately.
- Single element: no updates needed.
- All elements equal: depends on strict vs non-strict inequality.
- Duplicate elements: handle carefully (strict vs non-strict).
- Large values: coordinate compression may be needed for BIT.
- Negative coordinates: shift to positive range or use coordinate compression.

15. Variations

CDQ for 4D Partial Order

- Nested CDQ: first CDQ on x, inside CDQ on y, BIT on z and w.
- Complexity: $O(n \log^3 n)$.

CDQ with Divide and Conquer on Segment Tree

- CDQ can be combined with segment tree for more complex constraints.
- Used when the condition involves ranges rather than specific values.

CDQ with Mo's Algorithm

- Rare but possible combination for very specific problems.

CDQ without BIT (for 2D)

- If only 2 dimensions, CDQ is unnecessary — just sort and use a variable.

16. Related Algorithms

Algorithm	Difference	When to Choose
Sort (1D)	$O(n \log n)$, simple	1D partial order
Sort + BIT (2D)	$O(n \log n)$	2D partial order
CDQ + BIT (3D)	$O(n \log^2 n)$	3D partial order
Nested CDQ (4D)	$O(n \log^3 n)$	4D partial order
Segment Tree	$O(\log n)$ per query	Online queries, dynamic updates
KD-Tree	$O(n^{1-1/k})$ per query	General k-D queries, online

17. Practice Problems

Easy

1. **Count Inversions** — GFG → 1D partial order (can be solved with simpler methods).
2. **2D Partial Order Count** — Custom → Sort by x, BIT by y.

Medium

3. **3D Partial Order** — Codeforces 1093E → CDQ divide and conquer for 3D counting.
4. **Divisible Set** — Codeforces 1616E → CDQ + DP.

Hard

5. **LIS in 3D** — BOJ 2568 → CDQ + DP for longest increasing subsequence in 3D.
6. **CDQ Divide and Conquer** — Many CP platforms → Full implementation of CDQ for 3D problems.
7. **4D Partial Order** — Codeforces 1582F2 → Nested CDQ for 4D.

18. Interview Explanation

"CDQ divide and conquer is a technique for solving offline multidimensional partial order problems, named after the Chinese competitive programmer Chen Danqi. It works by using the index as the first dimension, dividing the array into two halves, recursively solving the left half, and then using the left half to update the right half. The key step is the combine phase, where we sort the elements by the second dimension and use a Fenwick tree to process the third dimension. This reduces a k-dimensional problem to a (k-1)-dimensional problem plus a divide-and-conquer step. The total complexity for 3D partial order is $O(n \log^2 n)$. CDQ is also powerful for offline DP where transitions depend on previous states."

19. Revision Notes

- CDQ: divide by index, solve left, combine (left→right), solve right.
- Combine step: sort by secondary dimension, use BIT for tertiary dimension.
- 3D partial order: $O(n \log^2 n)$.
- CDQ + DP: left updates BIT with DP values, right queries BIT.
- Always reset BIT after combine step.
- Sort by primary dimension first, update IDs.
- Only works offline (all data known in advance).

20. Final Cheat Sheet

Aspect	Detail
When to use	3D+ partial order, offline DP with constraints, multidimensional range queries
Main operations	Divide by index, sort by 2nd dim, BIT for 3rd dim, Combine left→right
Complexity	Time: $O(n \log^2 n)$ for 3D, Space: $O(n)$
Key code idea	<code>cdq(l,mid) → combine(l,r,mid) → cdq(mid+1,r)</code>

Aspect	Detail
Edge cases	Single element, duplicates, large values (compress), strict vs non-strict
Common mistake	Not resetting BIT, wrong recursion order, not handling equal elements

Final Notes

How to Use This Guide

1. **For interviews:** Read the "Interview Explanation" section for each algorithm to get a concise, confident answer.
2. **For coding practice:** Implement the C++ code from scratch. Then solve the practice problems.
3. **For revision:** Use the "Revision Notes" and "Final Cheat Sheet" for each algorithm.
4. **For CP contests:** Focus on the "Common Patterns" section to quickly identify problem types.

Master List of All Algorithms

Algorithm	Core Idea	Complexity	Key Technique
Merge Sort	Split, sort, merge	$O(n \log n)$	Merging sorted halves
Quick Sort	Pick pivot, partition, recurse	$O(n \log n)$ avg	Lomuto/Hoare partition
Binary Search	Divide search space by half	$O(\log n)$	Monotonic predicate
Count Inversions	Merge sort + count cross inversions	$O(n \log n)$	<code>inv += (L.size() - i)</code>
Max Subarray (D&C)	Max of left, right, cross	$O(n \log n)$	Best suffix + prefix at mid
Closest Pair	Strip of width $2d$, sort by y	$O(n \log n)$	Check only 7 points per point
Karatsuba	3 multiplications instead of 4	$O(n^{1.585})$	<code>(a+b)(c+d) - ac - bd</code>
Fast Exponentiation	Square and multiply by bits	$O(\log b)$	Binary representation of exponent
FFT	Roots of unity, butterfly	$O(n \log n)$	Point-value multiplication
CDQ	Divide by index, combine left→right	$O(n \log^2 n)$	Reduce k -D to $(k-1)$ -D

Good luck with your placements and competitive programming!